PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE - 5th Semester

Aug – Dec 2026

UE23CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT ON

ResolveX: Campus Infrastructure Incident, Maintenance and Asset Management System

Submitted by:

SRN	Name	Section
PES1UG24AM908	SONU AKASH	5D
PES1UG24AM225	RISHIT V N S	5D

Class of Prof. ***RAGHU B***

LAB FACULTY: DEVNA SUMESH

Abstract

In our college, complaints about broken ACs, projectors, Wi-Fi and lifts usually reach the maintenance office through phone calls, WhatsApp messages or a register. Because of this it is hard to track them, hard to decide which one should be fixed first, and nobody notices when ten students are complaining about the same problem in different words. The same machines also keep breaking down, since no one looks at their repair history.

We wanted the database itself to handle most of this work. When a student reports a problem, PostgreSQL calculates a priority score out of 100 using seven factors: how serious it is, how many people are affected, how critical the machine is, how important the building is, how often it has failed before, how urgent that category usually is, and whether it has been escalated. Based on this score the complaint gets a level from P1 to P4 and a deadline taken from an SLA table. We also convert every complaint into a 384-number sentence embedding and store it using pgvector. Because of this, a new report saying “Lab is very hot” gets matched with an existing “AC not cooling” report even though the two do not share any words, and the student can join that report instead of creating a duplicate.

Assigning work is done by a stored procedure that picks the technician who has the required skill and the fewest active jobs, and it uses row locks so two managers cannot assign the same job at once. Completing a repair happens inside a single transaction. It reduces the spare-part stock, saves the parts and labour cost, marks the incident as resolved and puts the machine back in service. If there is not enough stock, the whole thing is rolled back and nothing is saved. Triggers take care of the status history, the audit log and notifications, and we used views and window functions for analytics such as the risk score of each machine and the average time between its failures.

For data that does not have a fixed structure we used MongoDB. When a technician inspects a machine, the readings depend on the type of machine, and across our 9 categories there are 38 different reading fields. In MongoDB each inspection document only stores the fields that apply to it, and we still validate every document with `$jsonSchema`. For security, access is checked in three places: the Next.js middleware, a role check in every API route, and 65 Row-Level Security policies inside PostgreSQL.

Summary of what we built

Area	Count
PostgreSQL tables (3NF)	20, with 30 foreign keys and 57 CHECK constraints
Functions / stored procedures / triggers	11 / 6 / 17
Views	9 (1 materialized)
Row-Level Security policies	65 on 18 tables
Advanced SQL queries	23
MongoDB collections / indexes / aggregation pipelines	3 / 8 / 7

Area	Count
Next.js pages / API routes / user roles	18 / 32 / 4
Automated tests	121 (53 SQL, 27 MongoDB, 41 API and security)

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification – Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	7
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram	11
5.	SQL DDL Statements	14
6.	SQL DML Statements	18
7.	NoSQL / Vector Database Commands	21
8.	Results a) Resulting Tables' Screenshots	26
9.	Next.js – Project Tree Structure	39
10.	Sample Screenshots of Application Code and Demo	41
11.	Evaluation Rubrics – Completion Status	66

1. Introduction

a) Problem Statement

ResolveX: Campus Infrastructure Incident, Maintenance and Asset Management System

ResolveX is a web application we built for handling maintenance problems on our campus. Students and staff use it to report anything that is broken, like an AC, a projector or the Wi-Fi. The database then works out how urgent the problem is, checks if someone has already reported the same thing, sends a technician with the right skills, and keeps a record of the repair, the parts used and the cost.

At present, complaints reach the maintenance office through phone calls, WhatsApp messages or a register. It is hard to track them, hard to decide which one to fix first, duplicate complaints go unnoticed, and nobody looks at the repair history of machines that keep failing. ResolveX moves this logic into the database: priority scoring, SLA deadlines, duplicate detection by meaning, safe technician assignment, one-transaction repair completion, audit logging and role-based security.

Item	Detail
Course	Database Management Systems (UE23CS351A), Experiential Learning Level 3
Institution	PES University, B.Tech AIML (2024 batch, section D)
Databases	PostgreSQL 16 (relational), pgvector (vector), MongoDB 7 (NoSQL documents)
Application	Next.js 15 web app with 4 user roles
Repository	github.com/SonuAkash-collab/resolvex

2. User Requirements Specification – Tech Stack

2.1 User requirements

User role	What the user needs from the system
Student / staff	Report a problem; see similar reports while typing and join one instead of creating a duplicate; track the status, priority and deadline of own reports; edit own report; receive notifications.
Technician	See own work orders, most urgent first; start, hold and complete a job; record the spare parts used; write inspection logs for the machine type.
Facility manager	See the priority queue; assign a technician automatically or manually; mark duplicates; follow stock, asset health and analytics; download reports.
Admin	Manage users and roles, buildings, rooms and the SLA policy; manage the vector store; read the audit log.

2.2 Tech stack

We used three kinds of storage, each for a different reason. PostgreSQL holds the main data that has to stay consistent, such as incidents, work orders, stock and costs. pgvector is an extension inside PostgreSQL that we used for searching complaints by meaning. MongoDB holds the inspection logs and notes, whose structure changes from one type of machine to another.

Layer	Technology	Version	Why it was chosen
Relational database	PostgreSQL	16	ACID transactions, foreign keys, triggers, procedures and Row-Level Security for incidents, work orders, stock and costs
Vector database	pgvector extension	Docker image pgvector/pgvector:pg16	vector(384) column, HNSW index and cosine distance <=>; similarity can be joined with location and status in one SQL query
NoSQL database	MongoDB	7.0	Inspection readings differ per category (38 fields across 9 types); flexible documents with `$jsonSchema` validation, TTL and multikey indexes

Layer	Technology	Version	Why it was chosen
Embedding model	all-MiniLM-L6-v2 via Transformers.js (`@huggingface/transformers`)	3.8.1	Runs locally on the server, free and private; 384-number sentence vectors
Web framework	Next.js (App Router) + React	15.5 / 19.2	Pages and API routes in one project, server-side database access
PostgreSQL driver	node-postgres (`pg`)	8.23	Connection pool, parameterised queries, transactions
MongoDB driver	mongodb	6.21	Official Node.js driver
Authentication	bcryptjs + jose (JWT)	3.0 / 6.2	bcrypt password hashes; signed JWT session in an httpOnly cookie
Configuration	dotenv	16.4	Secrets kept in `.env.local`, not in Git
Containers	Docker Compose	—	Starts PostgreSQL+pgvector and MongoDB with one command
Runtime	Node.js	20 or newer	Runs Next.js, setup scripts and tests
Version control	Git and GitHub	—	Shared repository, commits from both members

2.3 Why this stack for our problem

Incidents, work orders, stock and cost must never contradict each other: stock cannot go negative, and a repair has to update stock, cost and status together. That needs foreign keys, CHECK constraints, triggers and ACID transactions, so the core is PostgreSQL. Duplicate complaints are written in very different words, so keyword search cannot match “Lab is very hot” with “AC not cooling”. Sentence embeddings can, and pgvector keeps them inside PostgreSQL, so similarity, location, status and the fix that worked come back in one SQL query without a second system. Next.js gives us pages and API routes in one project, and the embedding model runs on our own server, so no paid API is needed.

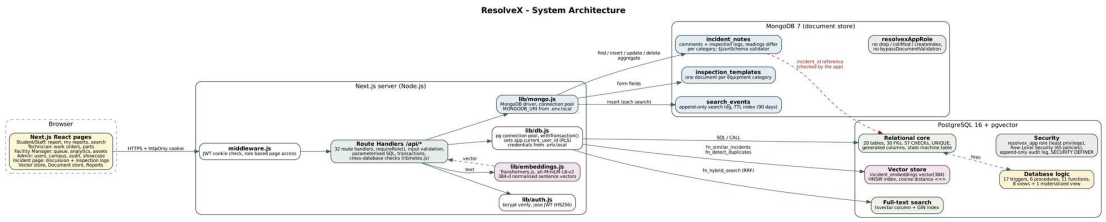
2.4 Why MongoDB (NoSQL) for our problem

An AC inspection records temperatures and refrigerant pressure, while a Wi-Fi inspection records signal strength and packet loss. Across our 9 categories there are 38 different reading fields. As SQL columns most of them would be empty for any given row, and every new

equipment type would need an ALTER TABLE. As MongoDB documents, each inspection stores only the fields of its own category, and a generated `$jsonSchema` validator still checks type and range. Comments and discussion threads are also naturally document-shaped, and the search log can expire automatically with a TTL index.

How a request flows through the system

1. The browser calls a Next.js page; the page calls an API route under `app/api/`.
2. The API route checks the session and role, then opens a PostgreSQL transaction and sets `app.current_user_id` so Row-Level Security knows the user.
3. Structured data, functions, procedures and vector search run in PostgreSQL; notes, inspection logs and the search log go to MongoDB.
4. When an incident is reported or edited, the server computes its embedding locally and stores it in the same transaction.



System architecture (`docs/diagrams/architecture.png`).

3. E-R Model

a) List of Entities [labeling Weak ones]

The ER model has 15 entities. Weak entities are marked in bold as WEAK.

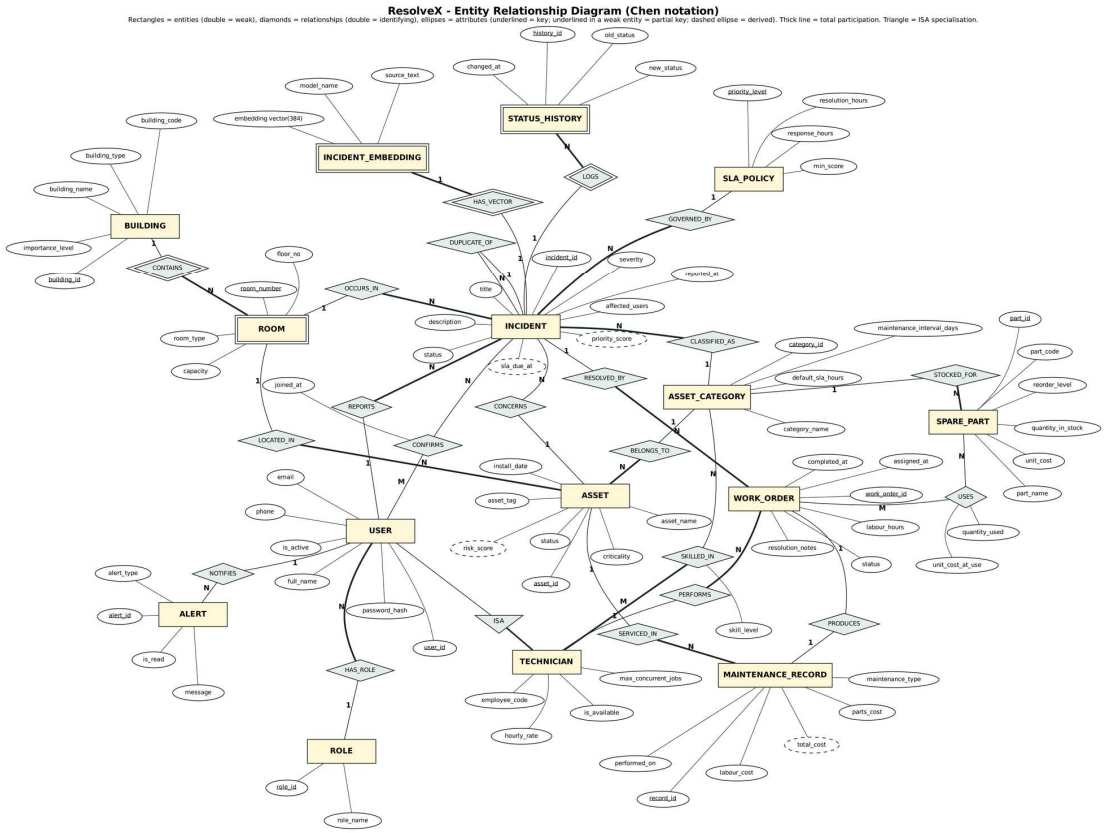
Entity	Type	Key	Notes
USER	Strong	user_id	email unique, bcrypt password hash
ROLE	Strong	role_id	4 roles
TECHNICIAN	Strong (ISA subclass of USER)	user_id (shared)	disjoint, partial specialisation
BUILDING	Strong	building_id	importance level feeds priority
ROOM	WEAK (owner BUILDING)	building + room_number	surrogate room_id with UNIQUE(building_id, room_number)
ASSET_CATEGORY	Strong	category_id	also used as incident category and skill area
ASSET	Strong	asset_id	derived attribute risk_score
INCIDENT	Strong	incident_id	derived attributes priority_score, priority_level, sla_due_at
SLA_POLICY	Strong	priority_level	P1 to P4
INCIDENT_EMBEDDING	WEAK (owner INCIDENT, 1:1)	incident_id	the vector store
STATUS_HISTORY	WEAK (owner INCIDENT)	incident + history_id	written by a trigger
WORK_ORDER	Strong	work_order_id	
SPARE_PART	Strong	part_id	
MAINTENANCE_RECORD	Strong	record_id	derived total_cost
ALERT	Strong	alert_id	written by triggers and procedures

b) List non-simple Attributes

Attribute	Entity	Kind	How it is handled
risk_score	ASSET	Derived	computed by `fn_asset_risk_score` and shown in `v_asset_health`

Attribute	Entity	Kind	How it is handled
priority_score	INCIDENT	Derived	stored; set by trigger `trg_incident_priority` from 7 factors
priority_level	INCIDENT	Derived	stored; looked up from `sla_policies` by the same trigger
sla_due_at	INCIDENT	Derived	stored; reported_at + SLA resolution hours (trigger)
search_vector	INCIDENT	Derived	generated tsvector column for full-text search
total_cost	MAINTENANCE_RECORD	Derived	`GENERATED ALWAYS AS` labour_cost + parts_cost
building + room_number	ROOM	Composite (partial key)	`UNIQUE(building_id, room_number)`
incident + history_id	STATUS_HISTORY	Composite (partial key)	history_id with FK to incidents
skills of a technician	TECHNICIAN	Multivalued	junction table `technician_skills` with skill_level
co-reporters of an incident	INCIDENT	Multivalued	junction table `incident_reporters` with joined_at
spare parts used	WORK_ORDER	Multivalued	junction table `work_order_parts`
tags, readings, checklist	incident_notes (MongoDB)	Multivalued / nested	arrays and sub-documents in MongoDB

c) ER Diagram Snapshot



ER diagram (Chen notation, `docs/diagrams/er_diagram.png`): 15 entities, 21 relationships, weak entities, ISA, M:N and a recursive relationship.

d) ERD from a tool like draw.io

The ER diagram above was generated with Graphviz from the script `docs/diagrams/make_diagrams.py` (output in PNG and SVG), using Chen notation: rectangles for entities, double rectangles for weak entities, diamonds for relationships and ovals for attributes. The relationships and their cardinalities are listed below.

Relationship	Between	Cardinality	Participation	Attributes
HAS_ROLE	USER – ROLE	N:1	total on user	
ISA	USER – TECHNICIAN	specialisation	partial	
SKILLED_IN	TECHNICIAN – ASSET_CATEGORY	M:N	total on technician	skill_level
CONTAINS (identifying)	BUILDING – ROOM	1:N	total on room	
LOCATED_IN	ASSET – ROOM	N:1	total on asset	
BELONGS_TO	ASSET – ASSET_CATEGORY	N:1	total on asset	
REPORTS	USER – INCIDENT	1:N	total on incident	

Relationship	Between	Cardinality	Participation	Attributes
CONFIRMS	USER – INCIDENT	M:N	partial	joined_at
OCCURS_IN	INCIDENT – ROOM	N:1	total on incident	
CONCERNS	INCIDENT – ASSET	N:1	partial	
CLASSIFIED_AS	INCIDENT – ASSET_CATEGORY	N:1	total on incident	
GOVERNED_BY	INCIDENT – SLA_POLICY	N:1	total on incident	
DUPLICATE_OF (recursive)	INCIDENT – INCIDENT	N:1	partial	
HAS_VECTOR (identifying)	INCIDENT – INCIDENT_EMBEDDING	1:1	total on embedding	
LOGS (identifying)	INCIDENT – STATUS_HISTORY	1:N	total on history	
RESOLVED_BY	INCIDENT – WORK_ORDER	1:N	total on work order	
PERFORMS	TECHNICIAN – WORK_ORDER	1:N	total on work order	
USES	WORK_ORDER – SPARE_PART	M:N	partial	quantity_used, unit_cost_at_use
STOCKED_FOR	SPARE_PART – ASSET_CATEGORY	N:1	total on part	
PRODUCES	WORK_ORDER – MAINTENANCE_RECORD	1:1	partial	
SERVICED_IN	ASSET – MAINTENANCE_RECORD	1:N	total on record	
NOTIFIES	ALERT – USER	N:1	partial	

4. Relational Model

a) List of Relations that are not Entities in ER Model

The relational schema has 20 tables. 15 of them come directly from the ER entities. The other 5 are not entities in the ER model:

Relation	Comes from	Purpose
`technician_skills`	M:N relationship SKILLED_IN	junction table, PK (technician_id, category_id), attribute skill_level
`incident_reporters`	M:N relationship CONFIRMS	junction table, PK (incident_id, user_id), attribute joined_at
`work_order_parts`	M:N relationship USES	junction table, PK (work_order_id, part_id), attributes quantity_used and unit_cost_at_use
`incident_status_transitions`	Design table	allowed (from_status, to_status) pairs, read by the status state-machine trigger
`audit_log`	System table	append-only JSONB history written by the audit trigger

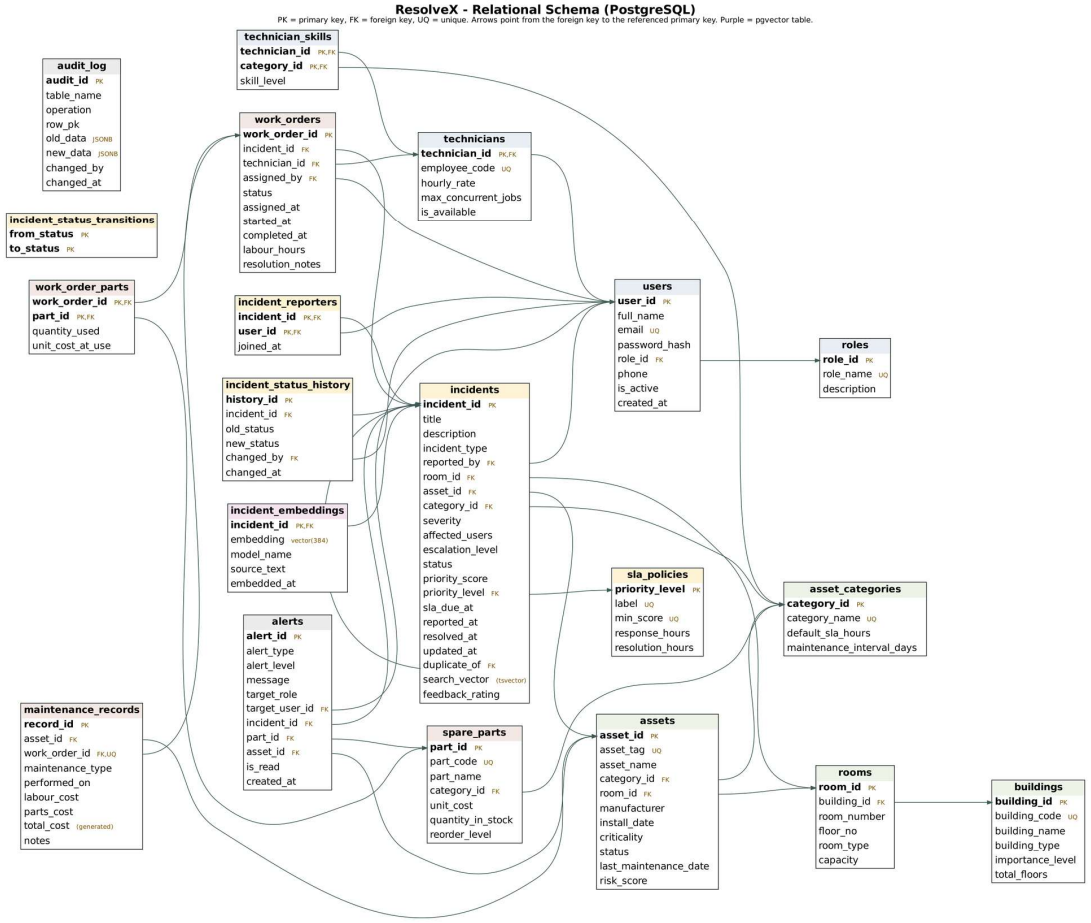
ER to relational mapping used: strong entity → own table; weak entity ROOM → table with the owner's key as FK; ISA → subclass table sharing the superclass PK; 1:N → FK on the N side; 1:1 → FK with UNIQUE; M:N → junction table; recursive relationship → self-referencing FK (`incidents.duplicate_of`); derived attributes → maintained by trigger, generated column or function.

Relational schema (PK = primary key, FK = foreign key, UQ = unique)

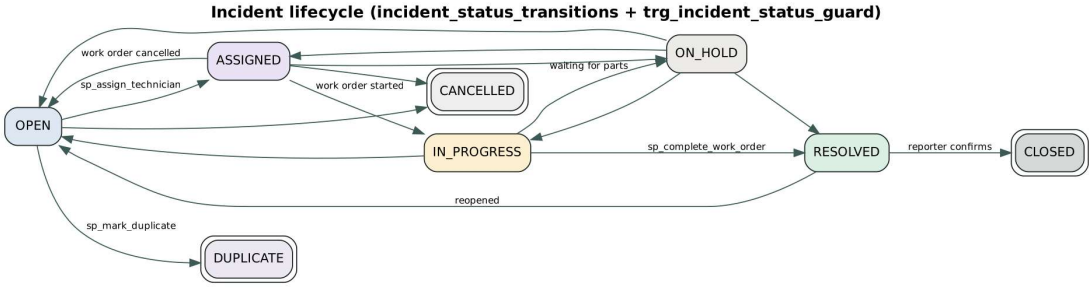
Relation	Primary key	Other columns and foreign keys
roles	role_id	role_name UQ, description
users	user_id	full_name, email UQ, password_hash, role_id FK→roles, phone, is_active, created_at
technicians	technician_id (FK→users)	employee_code UQ, hourly_rate, max_concurrent_jobs, is_available
technician_skills	technician_id FK→technicians, category_id FK→asset_categories	skill_level
buildings	building_id	building_code UQ, building_name, building_type, importance_level, total_floors
rooms	room_id	building_id FK→buildings, room_number, floor_no, room_type, capacity, UQ(building_id, room_number)
asset_categories	category_id	category_name UQ, default_sla_hours, maintenance_interval_days
assets	asset_id	asset_tag UQ, asset_name, category_id FK, room_id FK, manufacturer, install_date, criticality, status, last_maintenance_date, risk_score

Relation	Primary key	Other columns and foreign keys
sla_policies	priority_level	label UQ, min_score UQ, response_hours, resolution_hours
incident_status_transitions	from_status, to_status	
incidents	incident_id	title, description, incident_type, reported_by FK→users, room_id FK→rooms, asset_id FK→assets, category_id FK, severity, affected_users, escalation_level, status, priority_score, priority_level FK→sla_policies, sla_due_at, reported_at, resolved_at, updated_at, duplicate_of FK→incidents, search_vector, feedback_rating
incident_embeddings	incident_id (FK→incidents)	embedding vector(384), model_name, source_text, embedded_at
incident_reporters	incident_id FK→incidents, user_id FK→users	joined_at
incident_status_history	history_id	incident_id FK→incidents, old_status, new_status, changed_by FK→users, changed_at
work_orders	work_order_id	incident_id FK, technician_id FK→technicians, assigned_by FK→users, status, assigned_at, started_at, completed_at, labour_hours, resolution_notes
spare_parts	part_id	part_code UQ, part_name, category_id FK, unit_cost, quantity_in_stock, reorder_level
work_order_parts	work_order_id FK→work_orders, part_id FK→spare_parts	quantity_used, unit_cost_at_use
maintenance_records	record_id	asset_id FK→assets, work_order_id FK UQ, maintenance_type, performed_on, labour_cost, parts_cost, total_cost (generated), notes
alerts	alert_id	alert_type, alert_level, message, target_role, target_user_id FK, incident_id FK, part_id FK, asset_id FK, is_read, created_at
audit_log	audit_id	table_name, operation, row_pk, old_data JSONB, new_data JSONB, changed_by, changed_at

b) Schema Diagram



Relational schema (`docs/diagrams/relational_schema.png`): the 20 tables with primary and foreign keys.



Incident lifecycle (`docs/diagrams/incident_lifecycle.png`): the status state machine enforced by the trigger and the `incident_status_transitions` table.

5. SQL DDL Statements

The whole database is created by numbered SQL scripts in the `database/` folder, which `npm run db:setup` runs one after another. All commands shown below are taken from these files.

File	Contents
`00_drop.sql`	`DROP ... IF EXISTS CASCADE`, so the setup can be re-run
`01_schema.sql`	`CREATE TABLE` for 20 tables with PK, FK, UNIQUE, NOT NULL, CHECK
`02_alter.sql`	`ALTER TABLE`: self-referencing FK, generated full-text column, ADD / RENAME / DROP COLUMN
`03_indexes.sql`	21 indexes: B-tree, composite, partial, partial unique, expression, GIN, HNSW
`04_functions_procedures.sql`	11 functions and 6 stored procedures
`05_triggers.sql`	17 triggers
`06_views.sql`	9 views, 1 of them materialized
`07_security.sql`	roles, GRANT / REVOKE, 65 Row-Level Security policies
`08_seed.sql`	sample data (171 incidents, May to Oct 2026)
`09_queries.sql`	23 documented queries
`10_viva_live_demo.sql`	ready-to-run demo commands for every topic

5.1 CREATE TABLE with constraints

```
CREATE TABLE incidents (
    incident_id      SERIAL                        PRIMARY KEY,
    title            VARCHAR(150)                  NOT NULL,
    description      TEXT                          NOT NULL,
    reported_by      INT                           NOT NULL REFERENCES users(user_id),
      room_id          INT           NOT NULL REFERENCES rooms(room_id),
      asset_id         INT           REFERENCES assets(asset_id) ON DELETE SET NULL,
      category_id      INT           NOT NULL REFERENCES asset_categories(category_id),
      severity         SMALLINT      NOT NULL,
      affected_users   INT           NOT NULL DEFAULT 1,
      status           VARCHAR(15)   NOT NULL DEFAULT 'OPEN',
      priority_score   NUMERIC(5,2) NOT NULL DEFAULT 0,        -- computed by trigger
      priority_level   CHAR(2)       REFERENCES sla_policies(priority_level),
      sla_due_at       TIMESTAMPTZ,                            -- computed by trigger
      reported_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
      resolved_at      TIMESTAMPTZ,
      CONSTRAINT chk_severity       CHECK (severity BETWEEN 1 AND 5),
      CONSTRAINT chk_inc_status     CHECK (status IN ('OPEN','ASSIGNED','IN_PROGRESS','ON_HOLD',
                                                      'RESOLVED','CLOSED','DUPLICATE','CANCELLED')),
      CONSTRAINT chk_resolved_time CHECK (resolved_at IS NULL OR resolved_at >= reported_at)
);
```

5.2 ALTER TABLE

```
-- self-referencing foreign key (recursive relationship DUPLICATE_OF)
ALTER TABLE incidents ADD COLUMN duplicate_of INT;
ALTER TABLE incidents ADD CONSTRAINT fk_incident_duplicate_of
    FOREIGN KEY (duplicate of) REFERENCES incidents(incident id) ON DELETE SET NULL;


-- generated column for full-text search
ALTER TABLE incidents ADD COLUMN search_vector tsvector
    GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(description, '')), 'B')
    ) STORED;
```

```
-- ADD / RENAME / DROP COLUMN
ALTER TABLE buildings ADD COLUMN temp_notes TEXT;
ALTER TABLE buildings RENAME COLUMN temp_notes TO remarks;
ALTER TABLE buildings DROP COLUMN remarks;
```

5.3 Indexes

```
CREATE INDEX idx_incidents_status_priority ON incidents(status, priority_score DESC); -- composite
CREATE INDEX idx_incidents_active_sla ON incidents(sla_due_at)
    WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS','ON_HOLD');                    -- partial
CREATE UNIQUE INDEX uq_one_active_work_order ON work_orders(incident_id)
    WHERE status IN ('ASSIGNED','IN_PROGRESS');                                     -- business rule
CREATE UNIQUE INDEX uq_users_email_lower ON users(lower(email));                    -- expression
CREATE INDEX idx_incidents_fts ON incidents USING GIN (search_vector);              -- full-text
```

5.4 View with a window function (mean time between failures)

```
WITH failures AS (
    SELECT i.asset_id, i.reported_at,
           i.reported_at - LAG(i.reported_at)
               OVER (PARTITION BY i.asset_id ORDER BY i.reported_at) AS gap
      FROM incidents i
     WHERE i.asset_id IS NOT NULL AND i.incident_type = 'REACTIVE'
)
...
SELECT asset_tag, failures_180d, mtbf_days, predicted_next_failure, risk_score
  FROM v_asset_health ORDER BY risk_score DESC LIMIT 5;


REFRESH MATERIALIZED VIEW CONCURRENTLY mv_monthly_incident_stats;
```

5.5 Function: priority level from the SLA table

```
CREATE OR REPLACE FUNCTION fn_priority_level(p_score NUMERIC)
RETURNS CHAR(2) LANGUAGE sql STABLE AS $$
    SELECT priority_level FROM sla_policies
     WHERE min_score <= p_score
     ORDER BY min_score DESC
     LIMIT 1;
$$;
```

The other functions are `fn_priority_breakdown` and `fn_priority_score` (7-factor score out of 100), `fn_asset_risk_score`, `fn_similar_incidents`, `fn_detect_duplicates`, `fn_hybrid_search`, `fn_current_app_user`, `fn_current_app_role`, `fn_login_lookup` and `fn_refresh_monthly_stats`.

5.6 Stored procedure: assign the best technician (row locks)

```
CREATE OR REPLACE PROCEDURE sp_assign_technician(
    p_incident_id INT, p_assigned_by INT,
    p_technician_id INT DEFAULT NULL, INOUT p_work_order_id INT DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE v_inc incidents%ROWTYPE; v_tech INT := p_technician_id;
BEGIN
    SELECT * INTO v_inc FROM incidents WHERE incident_id = p_incident_id FOR UPDATE;
    IF v_inc.status <> 'OPEN' THEN
        RAISE EXCEPTION 'Only OPEN incidents can be assigned';
    END IF;
    IF v_tech IS NULL THEN
        SELECT t.technician_id INTO v_tech
          FROM technicians t
          JOIN technician_skills s ON s.technician_id = t.technician_id
                                  AND s.category_id = v_inc.category_id
          LEFT JOIN LATERAL (SELECT COUNT(*) AS active_jobs FROM work_orders w
                             WHERE w.technician_id = t.technician_id
                               AND w.status IN ('ASSIGNED','IN_PROGRESS')) wl ON TRUE
         WHERE t.is_available AND wl.active_jobs < t.max_concurrent_jobs
         ORDER BY wl.active_jobs ASC, s.skill_level DESC
         LIMIT 1 FOR UPDATE OF t;
```

```
    END IF;
    INSERT INTO work orders (incident id, technician id, assigned by)
    VALUES (p_incident_id, v_tech, p_assigned_by) RETURNING work_order_id INTO p_work_order_id;
    UPDATE incidents SET status = 'ASSIGNED' WHERE incident_id = p_incident_id;
END; $$;


CALL sp_assign_technician(168, 2, NULL, NULL);
```

The other procedures are `sp_complete_work_order`, `sp_mark_duplicate`, `sp_escalate_overdue`, `sp_schedule_preventive_maintenance` and `sp_restock_part`.

5.7 Trigger: automatic priority and deadline

```
CREATE OR REPLACE FUNCTION trgfn_incident_priority()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_hours SMALLINT;
BEGIN
    NEW.priority_score := fn_priority_score(NEW.severity, NEW.affected_users, NEW.asset_id,
                          NEW.room_id, NEW.category_id, NEW.escalation_level, NEW.reported_at,
                          CASE WHEN TG_OP = 'UPDATE' THEN NEW.incident_id END);
    NEW.priority_level := fn_priority_level(NEW.priority_score);
    SELECT resolution_hours INTO v_hours FROM sla_policies WHERE priority_level = NEW.priority_level;
    NEW.sla_due_at := NEW.reported_at + make_interval(hours => v_hours);
    RETURN NEW;
END; $$;


CREATE TRIGGER trg_incident_priority
BEFORE INSERT OR UPDATE OF severity, affected_users, asset_id, room_id, category_id, escalation_level
ON incidents FOR EACH ROW EXECUTE FUNCTION trgfn_incident_priority();
```

Other triggers: validation of asset, room and category; a status state machine; status history; co-reporter counting; technician skill and capacity checks; work order to incident and asset synchronisation; low-stock alerts; and a generic JSONB audit trigger on 6 tables.

5.8 Security: roles, privileges and Row-Level Security

```
CREATE ROLE resolvex_app NOLOGIN;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO resolvex app;
REVOKE UPDATE, DELETE ON audit_log FROM resolvex_app;          -- audit log is append-only


ALTER TABLE incidents ENABLE ROW LEVEL SECURITY;


CREATE POLICY p_incidents_update ON incidents FOR UPDATE
    USING (
        (SELECT fn_current_app_role()) IN ('FACILITY_MANAGER','ADMIN')
        OR reported_by = (SELECT fn_current_app_user())
        OR EXISTS (SELECT 1 FROM work_orders w
                    WHERE w.incident_id = incidents.incident_id
                      AND w.technician_id = (SELECT fn_current_app_user())));


CREATE POLICY p_incidents_delete ON incidents FOR DELETE
    USING ((SELECT fn_current_app_role()) = 'ADMIN');


-- test as a student: cannot change other people's incidents
SET ROLE resolvex_app;
BEGIN;
SELECT set_config('app.current_user_id', '11', true);
UPDATE incidents SET severity = 1 WHERE reported_by <> 11;   -- UPDATE 0
ROLLBACK;
RESET ROLE;
```

6. SQL DML Statements

6.1 INSERT, UPDATE, DELETE, SELECT

```
INSERT INTO buildings (building_code, building_name, building_type, importance_level, total_floors)
VALUES ('SPT', 'Sports Complex', 'SPORTS', 2, 2);


INSERT INTO rooms (building_id, room_number, floor_no, room_type, capacity)
VALUES ((SELECT building_id FROM buildings WHERE building_code = 'SPT'), 'GYM', 0, 'HALL', 80);


UPDATE incidents SET severity = 5 WHERE incident_id = 162;     -- trigger recomputes priority


DELETE FROM incident_embeddings WHERE incident_id = 167;       -- vector delete
```

6.2 Constraints rejecting bad data (each gives an error)

```
UPDATE spare_parts SET quantity_in_stock = -5 WHERE part_id = 1;     -- CHECK: stock >= 0
INSERT INTO rooms (building_id, room_number, floor_no, room_type)
VALUES (999, 'X1', 0, 'OFFICE');                                     -- FOREIGN KEY violation
INSERT INTO buildings (building_code, building_name, building_type, total_floors)
VALUES ('LIB', 'Duplicate code', 'LIBRARY', 1);                      -- UNIQUE violation
DELETE FROM spare_parts WHERE part_id = 1;                           -- FK: part used in repairs
```

6.3 Join, GROUP BY, HAVING, recursive CTE, window function and ROLLUP

```
-- JOIN + GROUP BY + HAVING
SELECT c.category_name, COUNT(*) AS incidents, ROUND(AVG(i.priority_score), 1) AS avg_priority
  FROM incidents i JOIN asset_categories c ON c.category_id = i.category_id
 GROUP BY c.category_name
HAVING COUNT(*) > 10
 ORDER BY incidents DESC;


-- RECURSIVE CTE: follow duplicate chains to the original incident
WITH RECURSIVE chain AS (
    SELECT incident_id, duplicate_of, incident_id AS root_id, 0 AS depth
      FROM incidents WHERE duplicate_of IS NULL
    UNION ALL
    SELECT i.incident_id, i.duplicate_of, c.root_id, c.depth + 1
      FROM incidents i JOIN chain c ON i.duplicate_of = c.incident_id
)
SELECT root_id AS original_incident, COUNT(*) - 1 AS merged_duplicates
  FROM chain GROUP BY root_id HAVING COUNT(*) > 1;


-- WINDOW function: top 3 most expensive assets per category
WITH asset_cost AS (
    SELECT c.category_name, a.asset_tag, SUM(m.total_cost) AS cost,
           RANK() OVER (PARTITION BY c.category_name ORDER BY SUM(m.total_cost) DESC) AS rnk
         FROM maintenance records m
         JOIN assets a ON a.asset_id = m.asset_id
         JOIN asset_categories c ON c.category_id = a.category_id
        GROUP BY c.category_name, a.asset_tag
)
SELECT * FROM asset_cost WHERE rnk <= 3;


-- ROLLUP subtotals
SELECT COALESCE(status, 'ALL STATUSES') AS status, COALESCE(priority_level, '--') AS priority, COUNT(*)
  FROM incidents GROUP BY ROLLUP (status, priority_level);
```

6.4 Nested and correlated subqueries

```
-- correlated subquery: NOT EXISTS (assets with no failure in the last 90 days)
SELECT a.asset_tag, a.asset_name, c.category_name, a.last_maintenance_date
  FROM assets a
  JOIN asset_categories c ON c.category_id = a.category_id
 WHERE a.status = 'OPERATIONAL'
   AND NOT EXISTS (SELECT 1
                     FROM incidents i
                    WHERE i.asset_id = a.asset_id
                      AND i.incident type = 'REACTIVE'
                      AND i.reported_at >= now() - INTERVAL '90 days')
```

```
 ORDER BY c.category_name, a.asset_tag;


-- correlated subquery in WHERE (incidents slower than twice their category average)
SELECT i.incident_id, i.title, c.category_name,
       ROUND(EXTRACT(EPOCH FROM (i.resolved_at - i.reported_at)) / 3600, 1) AS hours_taken
  FROM incidents i
  JOIN asset_categories c ON c.category_id = i.category_id
 WHERE i.resolved_at IS NOT NULL
   AND (i.resolved_at - i.reported_at) > 2 * (
        SELECT AVG(i2.resolved_at - i2.reported_at)
          FROM incidents i2
         WHERE i2.category_id = i.category_id AND i2.resolved_at IS NOT NULL)
 ORDER BY hours_taken DESC
 LIMIT 10;


-- nested subquery: NOT IN (rooms that never reported an incident)
SELECT b.building_name, r.room_number, r.room_type
  FROM rooms r
  JOIN buildings b ON b.building_id = r.building_id
 WHERE r.room_id NOT IN (SELECT DISTINCT room_id FROM incidents)
 ORDER BY b.building_name, r.room_number;
```

6.5 Transaction: completing a repair (all or nothing)

`sp_complete_work_order` locks the work order and each spare part with `FOR UPDATE`, checks stock, records parts at today's price, deducts stock, inserts the maintenance record with labour and parts cost, and marks the job COMPLETED. Any error rolls back every step.

```
-- too many parts: ERROR "Insufficient stock", nothing is saved
CALL sp_complete_work_order(156, 4, 2, 'Replaced the run capacitor', '[{"part_id": 2, "quantity": 50}]');


-- correct quantity: stock, cost, incident and asset updated together
CALL sp_complete_work_order(156, 4, 2, 'Replaced the run capacitor', '[{"part_id": 2, "quantity": 1}]');


-- manual transaction with ROLLBACK and SAVEPOINT
BEGIN;
UPDATE spare_parts SET unit_cost = unit_cost + 10 WHERE part_id = 7;
SAVEPOINT before_mistake;
UPDATE spare_parts SET unit_cost = 0 WHERE part_id = 7;     -- violates CHECK
ROLLBACK TO SAVEPOINT before mistake;                       -- undo only the mistake
COMMIT;
```

7. NoSQL / Vector Database Commands

Apart from the relational tables, our project uses two non-relational stores. pgvector stores the embeddings that let us search complaints by meaning, and MongoDB stores the inspection logs and notes. Both can be created, read, updated and deleted from the web app.

7.1 Vector database (pgvector inside PostgreSQL)

For every complaint we join the title, description and category into one sentence and pass it to the all-MiniLM-L6-v2 model, which gives back a list of 384 numbers (the embedding). The model runs on our own Next.js server, so we did not need any paid API. Sentences with a similar meaning end up with vectors that point in a similar direction, which is why “Lab is very hot” finds “AC not cooling”.

Enable the extension, create the vector table and index

```
CREATE EXTENSION IF NOT EXISTS vector;


CREATE TABLE incident_embeddings (
    incident_id   INT          PRIMARY KEY REFERENCES incidents(incident_id) ON DELETE CASCADE,
    embedding     vector(384) NOT NULL,
    model_name    VARCHAR(60) NOT NULL DEFAULT 'Xenova/all-MiniLM-L6-v2',
    source_text   TEXT         NOT NULL,
    embedded_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);


CREATE INDEX idx_incident_embeddings_hnsw
    ON incident_embeddings USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

CREATE: store the vector in the same transaction as the incident (`app/api/incidents/route.js`)

```
INSERT INTO incident_embeddings (incident_id, embedding, model_name, source_text)
VALUES ($1, $2::vector, $3, $4);
```

READ: similarity search (`fn_similar_incidents`; `<=>` is cosine distance, similarity = 1 − distance)

```
WITH nearest AS (
     SELECT e.incident_id, 1 - (e.embedding <=> p_embedding) AS sim
       FROM incident_embeddings e
      ORDER BY e.embedding <=> p_embedding       -- uses the HNSW index
      LIMIT p_limit * 4 + 5
)
SELECT i.incident_id, i.title, b.building_name, r.room_number, ROUND(n.sim::NUMERIC, 4) AS similarity
   FROM nearest n
   JOIN incidents i ON i.incident_id = n.incident_id
   JOIN rooms r     ON r.room_id = i.room_id
   JOIN buildings b ON b.building_id = r.building_id
  WHERE n.sim >= p_min_similarity
    AND i.status NOT IN ('DUPLICATE','CANCELLED')
  ORDER BY n.sim DESC LIMIT p_limit;


-- the 5 incidents most similar in meaning to incident 168
SELECT i.incident_id, i.title,
       ROUND((1 - (e.embedding <=> (SELECT embedding FROM incident_embeddings WHERE incident_id =
168)))::NUMERIC, 3) AS similarity
  FROM incident_embeddings e JOIN incidents i ON i.incident_id = e.incident_id
 WHERE e.incident_id <> 168
 ORDER BY e.embedding <=> (SELECT embedding FROM incident_embeddings WHERE incident_id = 168)
 LIMIT 5;


-- read the stored numbers and check the vector
SELECT incident_id, vector_dims(embedding) AS dims, (embedding::real[])[1:5] AS first_values,
       round(vector_norm(embedding)::numeric, 3) AS norm
```

```
  FROM incident_embeddings WHERE incident_id = 167;
```

`fn_detect_duplicates` runs while a student types a report: it compares against active incidents from the last 14 days and adds +0.15 for the same asset, +0.10 for the same room and +0.05 for the same building. `fn_hybrid_search` combines vector search with full-text search using Reciprocal Rank Fusion, score = 1 / (60 + rank).

UPDATE: re-embed when the text changes (`app/api/incidents/[id]/route.js`)

```
UPDATE incident_embeddings
   SET embedding = $2::vector, source_text = $3, model_name = $4, embedded_at = now()
 WHERE incident_id = $1;
```

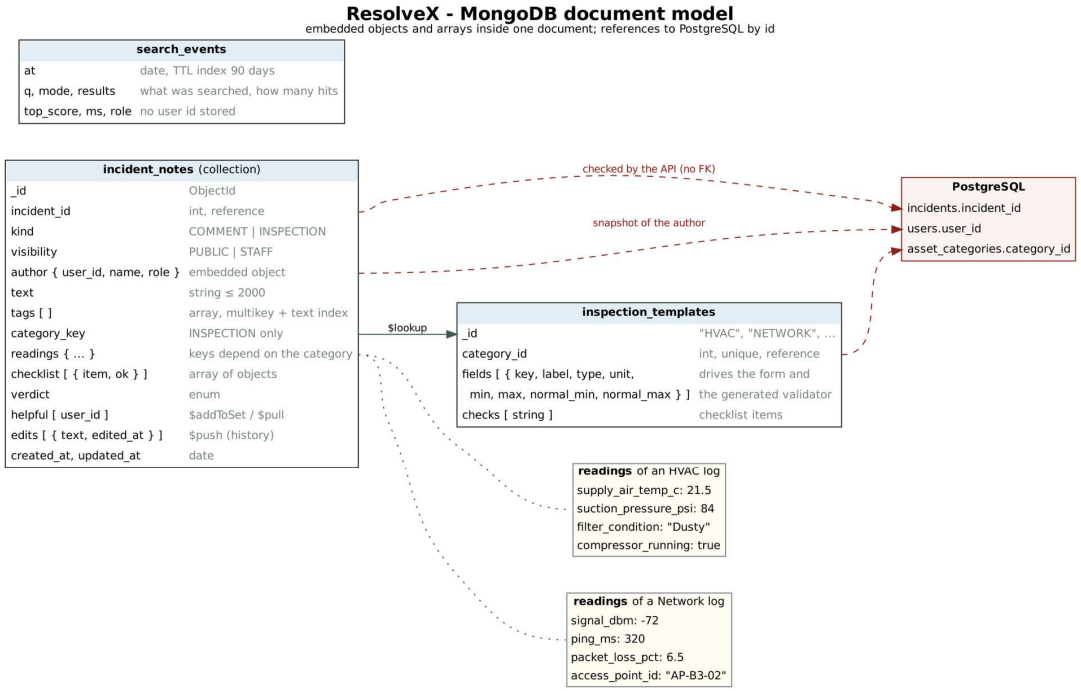
DELETE: remove a vector, admin only (`app/api/vectors/[id]/route.js`, protected by an RLS policy)

```
DELETE FROM incident_embeddings WHERE incident_id = $1;
```

Operation	Where in the app
Create	Report a problem: incident and vector saved in one transaction; Vector store → Create missing
Read	Duplicate panel while typing, Semantic search page, Vector store → View
Update	Edit title and description → Save and re-embed; Vector store → Re-embed
Delete	Vector store → Delete (admin only)

7.2 NoSQL database (MongoDB)

Collection	Holds	Indexes
`inspection_templates`	one document per equipment category: its reading fields, ranges and checklist	unique on `category_id`
`incident_notes`	comments and technicians' inspection logs, linked to PostgreSQL by `incident_id`	compound, nested-field, multikey (`tags`), partial (inspections only), text
`search_events`	log of every search; deleted automatically after 90 days	TTL, compound



Document model (`docs/diagrams/document_model.png`).

Create collections with $jsonSchema validators and indexes (`scripts/setup-mongo.mjs`)

```
const strict = { validationLevel: "strict", validationAction: "error" };
await db.createCollection("inspection_templates", { validator: templatesValidator, ...strict });
await db.createCollection("incident_notes",       { validator: notesValidator(TEMPLATES), ...strict });
await db.createCollection("search_events",        { validator: searchEventsValidator, ...strict });


db.incident_notes.createIndex({ incident_id: 1, created_at: -1 }, { name: "ix_incident_thread" });
db.incident_notes.createIndex({ tags: 1 }, { name: "ix_tags" });                          // multikey
db.incident_notes.createIndex({ category_key: 1, verdict: 1 },
    { name: "ix_inspection_verdict", partialFilterExpression: { kind: "INSPECTION" } }); // partial
db.incident_notes.createIndex({ text: "text", tags: "text" }, { weights: { text: 1, tags: 3 } });
db.search_events.createIndex({ at: 1 }, { name: "ttl_search_events", expireAfterSeconds: 90 * 86400 });
```

We generate the validator for `incident_notes` from the templates, so an HVAC inspection can only contain HVAC readings, and each reading must have the correct type and range (`additionalProperties: false`). A document also has to match exactly one shape (`oneOf`): either a normal comment, or an inspection of one category.

Least-privilege login for the app

```
db.createRole({ role: "resolvexAppRole", roles: [], privileges: [
  { resource: { db: "resolvex", collection: "incident_notes" },       actions: ["find", "insert",
"update", "remove"] },
  { resource: { db: "resolvex", collection: "search_events" },        actions: ["find", "insert"] }, //
append-only
  { resource: { db: "resolvex", collection: "inspection_templates" }, actions: ["find"] } // read-only
]});
db.createUser({ user: "resolvex_app", pwd: "<from .env.local>", roles: [{ role: "resolvexAppRole", db:
"resolvex" }] });
```

CREATE

```
db.incident_notes.insertOne({
  incident_id: 168, kind: "INSPECTION", visibility: "STAFF",
```

```
  author: { user_id: 4, name: "Ravi Kumar", role: "TECHNICIAN" },
  text: "Filter blocked and refrigerant low", category key: "HVAC",
  readings: { supply_air_temp_c: 21.5, suction_pressure_psi: 84, filter_condition: "Dusty" },
  checklist: [{ item: "Filter clean", ok: false }], verdict: "NEEDS_PARTS", created_at: new Date()
});


// rejected by the validator: an Electrical reading in an HVAC log
db.incident_notes.insertOne({ incident_id: 168, kind: "INSPECTION", category_key: "HVAC",
  readings: { voltage_v: 230 }, ... });   // MongoServerError: Document failed validation
```

READ

```
db.incident_notes.find({ incident_id: 168, visibility: "PUBLIC" }).sort({ created_at: -1 });      //
thread
db.incident_notes.find({ "readings.suction_pressure_psi": { $lt: 100 } });                        // nested
field
db.incident_notes.countDocuments({ checklist: { $elemMatch: { item: "Drain line clear", ok: false } } });
db.incident_notes.find({ $text: { $search: "fan motor noise" } }, { score: { $meta: "textScore" } });
```

UPDATE

```
db.incident_notes.updateOne({ _id: id },
  { $set: { text: "new text", updated_at: new Date() },
    $push: { edits: { text: "old text", edited_at: new Date() } } });


db.incident_notes.updateOne({ _id: id }, { $addToSet: { helpful: 4 } });                  // vote, no
duplicates
db.incident_notes.updateOne({ _id: id }, { $pull: { helpful: 4 } });                      // remove vote
```

DELETE

```
db.incident_notes.deleteOne({ _id: id, "author.user_id": 11 });
```

Aggregation pipeline (one of the 7 MongoDB reports): most failed checklist items per category

```
db.incident_notes.aggregate([
  { $match: { kind: "INSPECTION" } },
  { $unwind: "$checklist" },
  { $match: { "checklist.ok": false } },
  { $group: { _id: { category: "$category_key", check: "$checklist.item" }, failed: { $sum: 1 } } },
  { $sort: { failed: -1 } },
  { $limit: 5 }
]);
```

Operation	Where in the app	Code
Create	Incident page → Discussion and field logs → comment or inspection log	`app/api/incidents/[id]/notes/route.js` (`insertOne`)
Read	Incident page notes thread; Document store page	same file (`find`)
Update	Edit a comment; Helpful vote	`app/api/notes/[noteId]/route.js` (`findOneAndUpdate`, `$push`, `$addToSet`, `$pull`)
Delete	Delete your own comment	same file (`deleteOne`)

8. Results

a) Resulting Tables' Screenshots

The screenshots below show the tables and query results produced by the database, as displayed by the running application with our sample data (171 incidents).

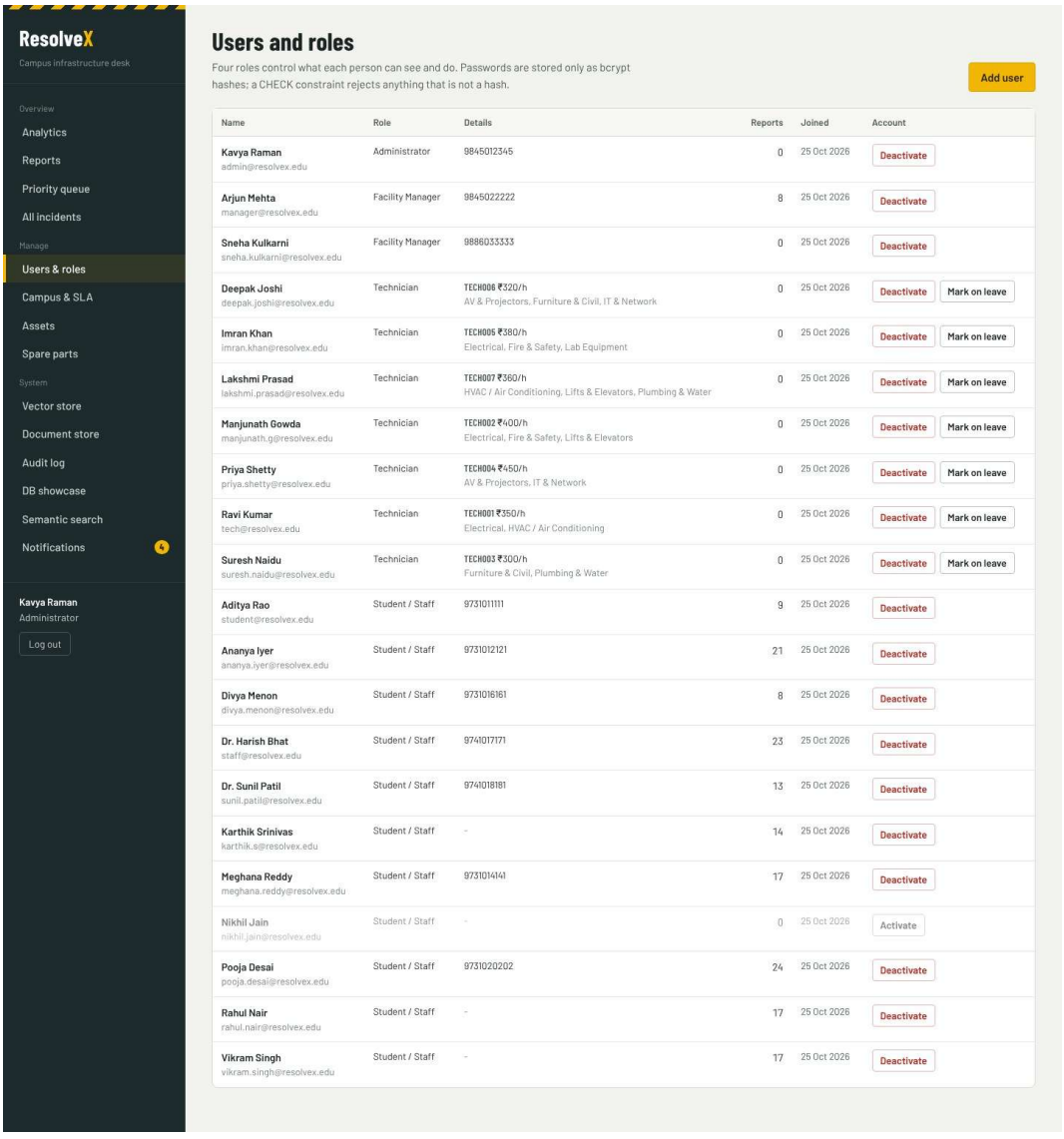


Figure 1. The `users` table with roles (admin page).

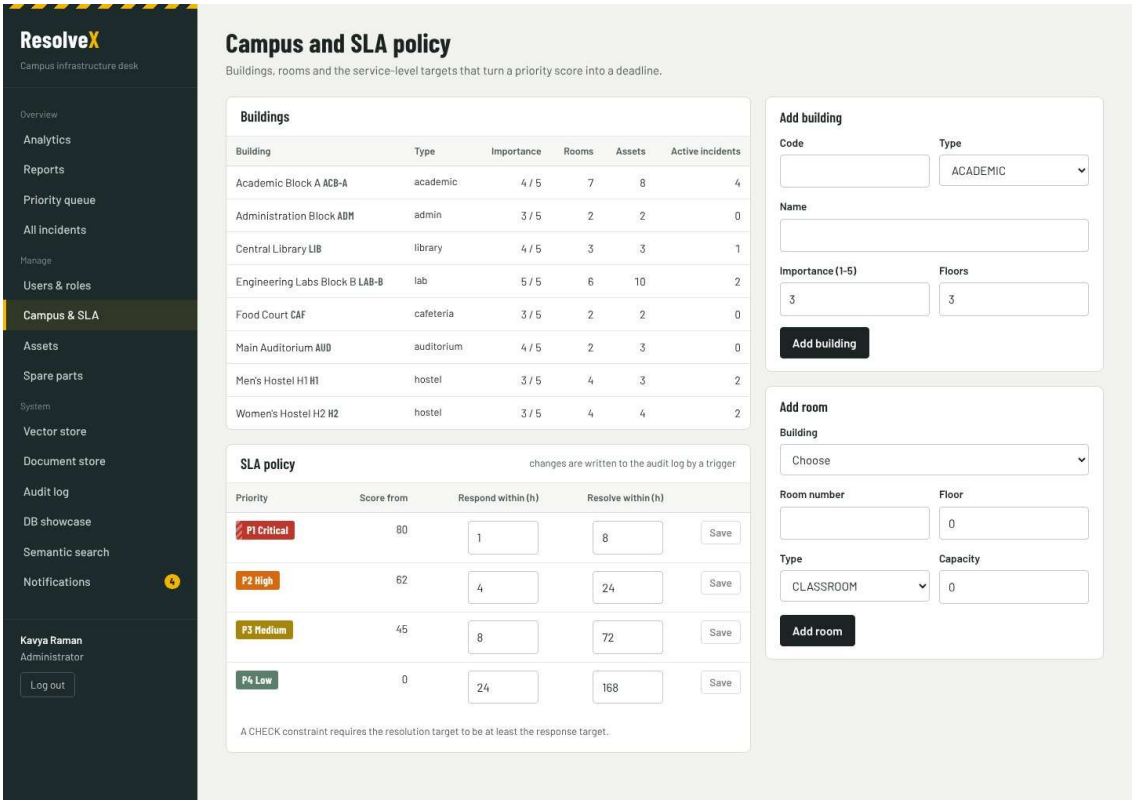


Figure 2. The `buildings`, `rooms` and `sla_policies` tables (admin page).

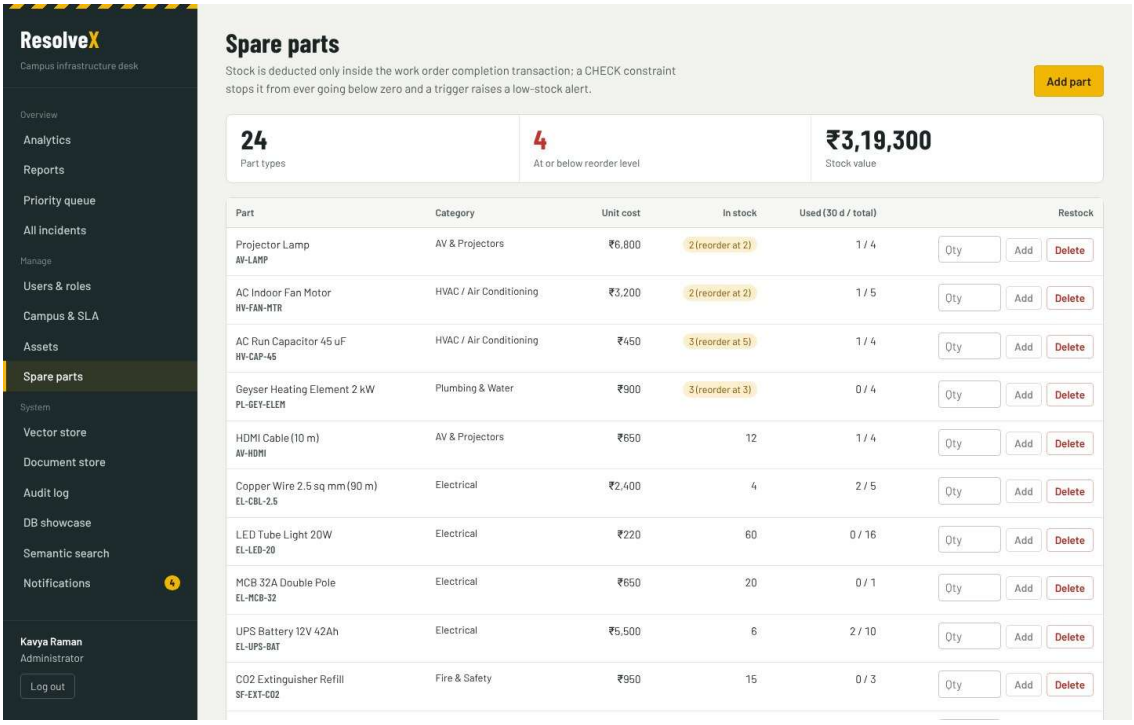


Figure 3. The `spare_parts` table with low-stock warnings and the restock option. (part 1 of 2)

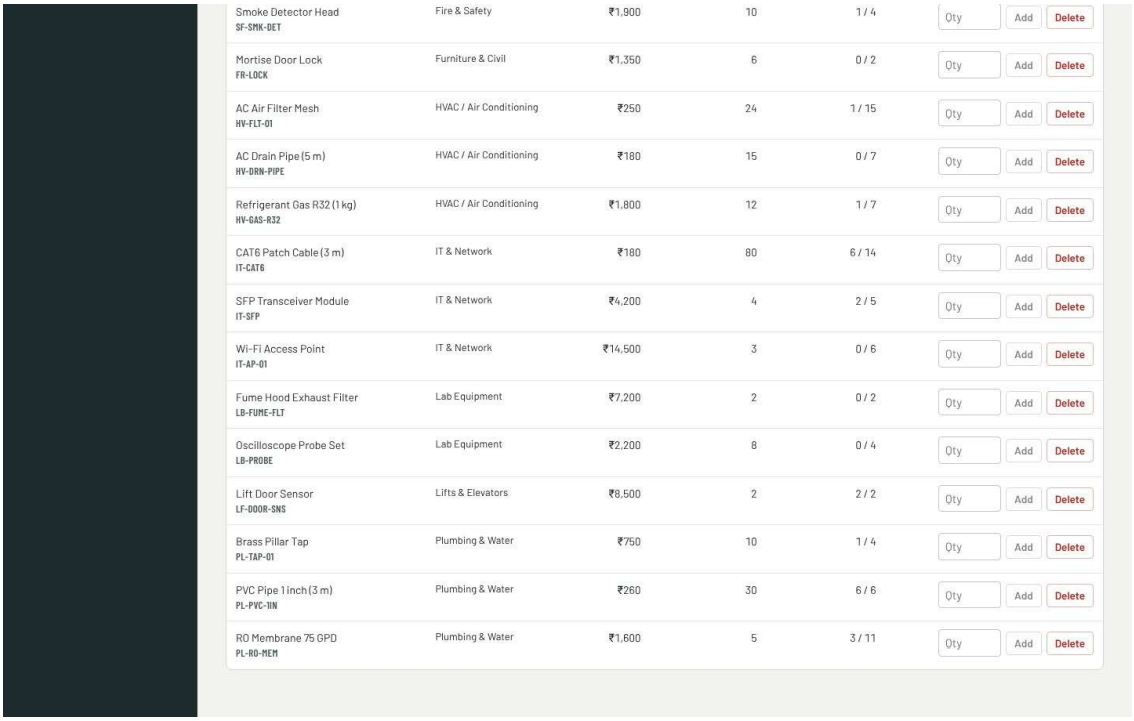


Figure 3. The `spare_parts` table with low-stock warnings and the restock option. (part 2 of 2)

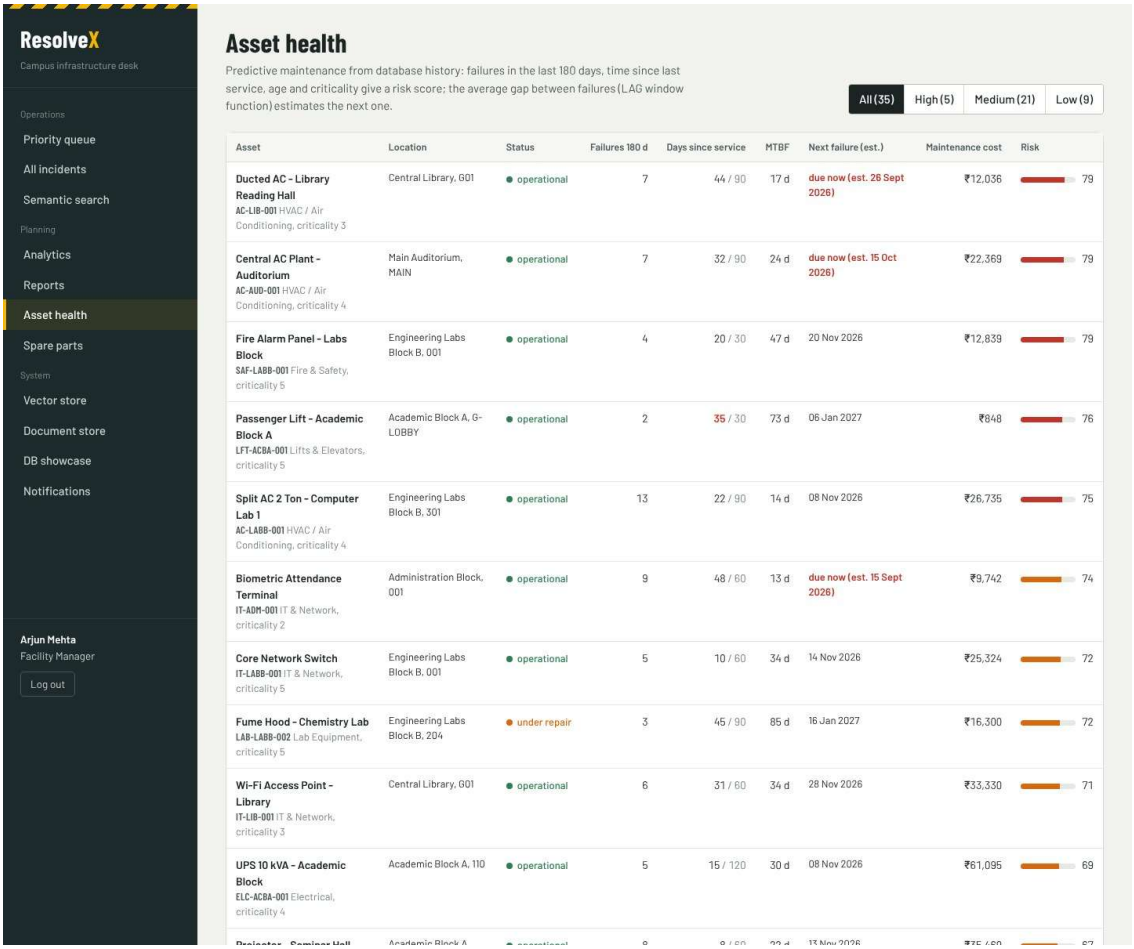
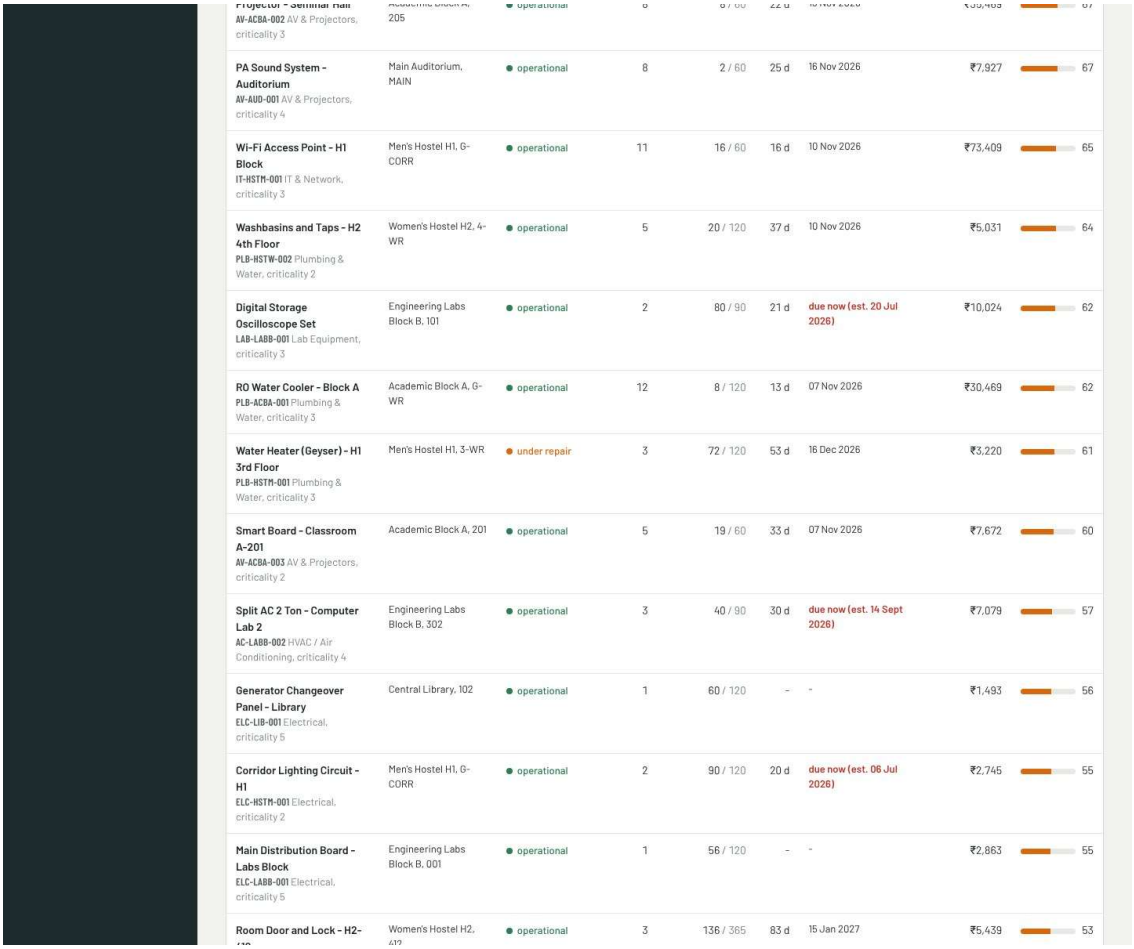
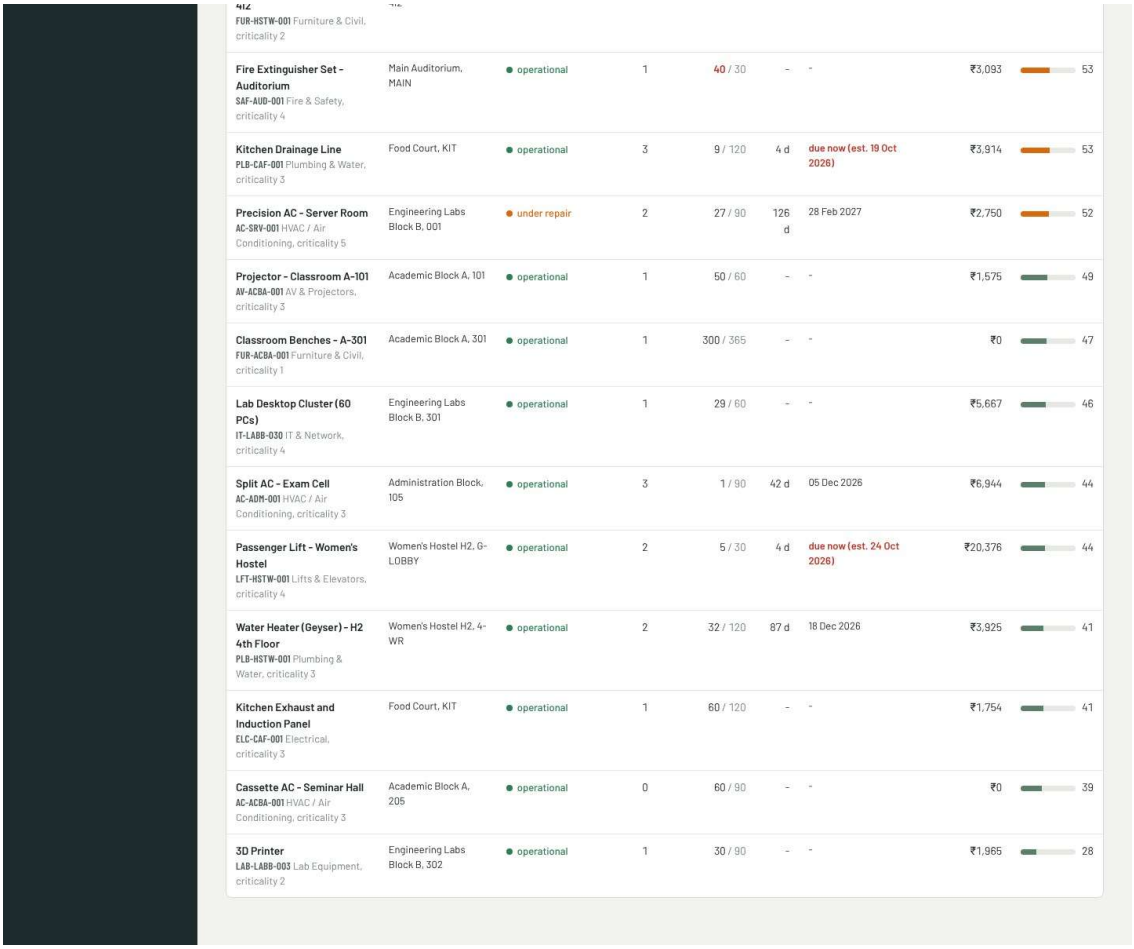


Figure 4. The `v_asset_health` view: each machine's risk score, the average time between failures (LAG window function) and the predicted next failure. (part 1 of 3)



Projector - Seminar Hall AV-ACBA-002 AV & Projectors, criticality 3	Academic Block A, 205	operational	8	8 / 60	22 d	16 Nov 2026	₹33,483	67
PA Sound System - Auditorium AV-AUD-001 AV & Projectors, criticality 4	Main Auditorium, MAIN	operational	8	2 / 60	25 d	16 Nov 2026	₹7,927	67
Wi-Fi Access Point - H1 Block IT-HSTH-001 IT & Network, criticality 3	Men's Hostel H1, G-CORR	operational	11	16 / 60	16 d	10 Nov 2026	₹73,409	65
Washbasins and Taps - H2 4th Floor PLB-HSTW-002 Plumbing & Water, criticality 2	Women's Hostel H2, 4-WR	operational	5	20 / 120	37 d	10 Nov 2026	₹5,031	64
Digital Storage Oscilloscope Set LAB-LABB-001 Lab Equipment, criticality 3	Engineering Labs Block B, 101	operational	2	80 / 90	21 d	due now (est. 20 Jul 2026)	₹10,024	62
RO Water Cooler - Block A PLB-ACBA-001 Plumbing & Water, criticality 3	Academic Block A, G-WR	operational	12	8 / 120	13 d	07 Nov 2026	₹30,469	62
Water Heater (Geyser) - H1 3rd Floor PLB-HSTH-001 Plumbing & Water, criticality 3	Men's Hostel H1, 3-WR	under repair	3	72 / 120	53 d	16 Dec 2026	₹3,220	61
Smart Board - Classroom A-201 AV-ACBA-003 AV & Projectors, criticality 2	Academic Block A, 201	operational	5	19 / 60	33 d	07 Nov 2026	₹7,672	60
Split AC 2 Ton - Computer Lab 2 AC-LABB-002 HVAC / Air Conditioning, criticality 4	Engineering Labs Block B, 302	operational	3	40 / 90	30 d	due now (est. 14 Sept 2026)	₹7,079	57
Generator Changeover Panel - Library ELC-LIB-001 Electrical, criticality 5	Central Library, 102	operational	1	60 / 120	-	-	₹1,493	56
Corridor Lighting Circuit - H1 ELC-HSTH-001 Electrical, criticality 2	Men's Hostel H1, G-CORR	operational	2	90 / 120	20 d	due now (est. 06 Jul 2026)	₹2,745	55
Main Distribution Board - Labs Block ELC-LABB-001 Electrical, criticality 5	Engineering Labs Block B, 001	operational	1	56 / 120	-	-	₹2,863	55
Room Door and Lock - H2-	Women's Hostel H2,	operational	3	136 / 365	83 d	15 Jan 2027	₹5,439	53

Figure 4. The `v_asset_health` view: each machine's risk score, the average time between failures (LAG window function) and the predicted next failure. (part 2 of 3)



412 FUR-HSTW-001 Furniture & Civil, criticality 2								
Fire Extinguisher Set - Auditorium SAF-AUD-001 Fire & Safety, criticality 4	Main Auditorium, MAIN	operational	1	40 / 30	-	-	₹3,093	53
Kitchen Drainage Line PLB-CAF-001 Plumbing & Water, criticality 3	Food Court, KIT	operational	3	9 / 120	4 d	due now (est. 19 Oct 2026)	₹3,914	53
Precision AC - Server Room AC-SRV-001 HVAC / Air Conditioning, criticality 5	Engineering Labs Block B, 001	under repair	2	27 / 90	126 d	28 Feb 2027	₹2,750	52
Projector - Classroom A-101 AV-ACBA-001 AV & Projectors, criticality 3	Academic Block A, 101	operational	1	50 / 60	-	-	₹1,575	49
Classroom Benches - A-301 FUR-ACBA-001 Furniture & Civil, criticality 1	Academic Block A, 301	operational	1	300 / 365	-	-	₹0	47
Lab Desktop Cluster (60 PCs) IT-LABB-030 IT & Network, criticality 4	Engineering Labs Block B, 301	operational	1	29 / 60	-	-	₹5,667	46
Split AC - Exam Cell AC-ADM-001 HVAC / Air Conditioning, criticality 3	Administration Block, 105	operational	3	1 / 90	42 d	05 Dec 2026	₹6,944	44
Passenger Lift - Women's Hostel LFT-HSTW-001 Lifts & Elevators, criticality 4	Women's Hostel H2, G-LOBBY	operational	2	5 / 30	4 d	due now (est. 24 Oct 2026)	₹20,376	44
Water Heater (Geyser) - H2 4th Floor PLB-HSTW-001 Plumbing & Water, criticality 3	Women's Hostel H2, 4-WR	operational	2	32 / 120	87 d	18 Dec 2026	₹3,925	41
Kitchen Exhaust and Induction Panel ELC-CAF-001 Electrical, criticality 3	Food Court, KIT	operational	1	60 / 120	-	-	₹1,754	41
Cassette AC - Seminar Hall AC-ACBA-001 HVAC / Air Conditioning, criticality 3	Academic Block A, 205	operational	0	60 / 90	-	-	₹0	39
3D Printer LAB-LABB-003 Lab Equipment, criticality 2	Engineering Labs Block B, 302	operational	1	30 / 90	-	-	₹1,965	28

Figure 4. The `v_asset_health` view: each machine's risk score, the average time between failures (LAG window function) and the predicted next failure. (part 3 of 3)

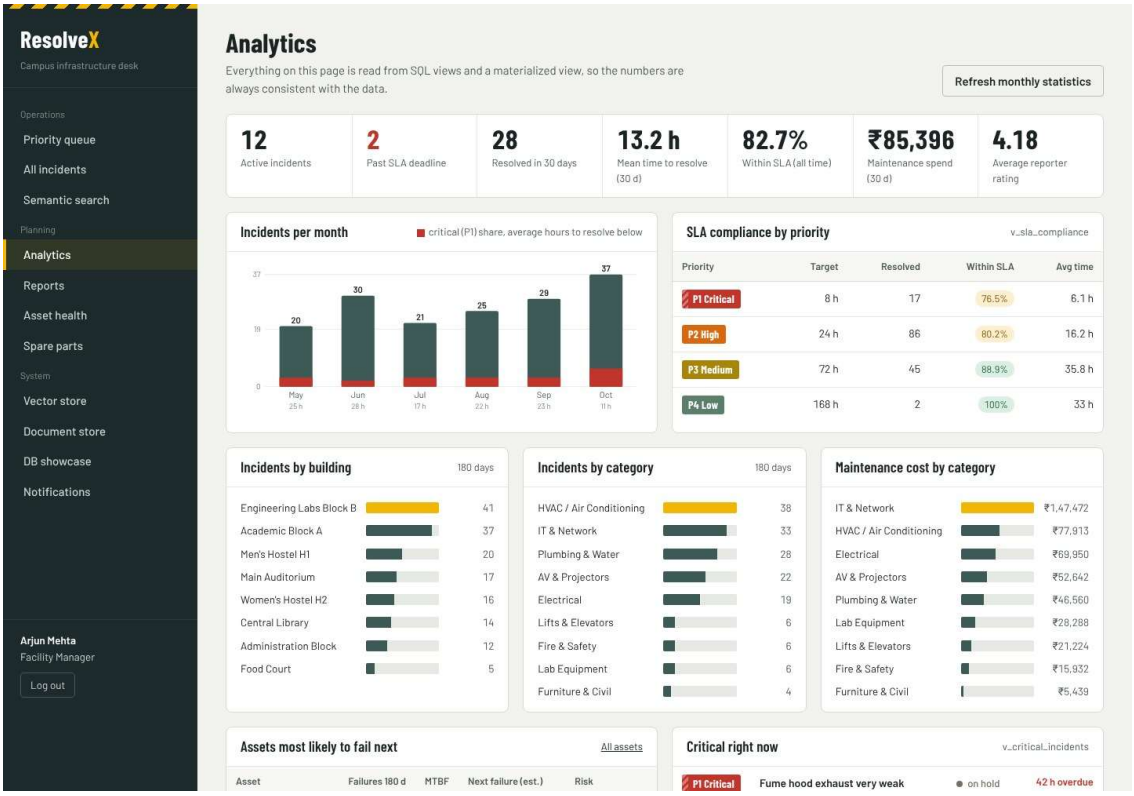


Figure 5. The analytics views and the materialized view `mv_monthly_incident_stats`. (part 1 of 2)

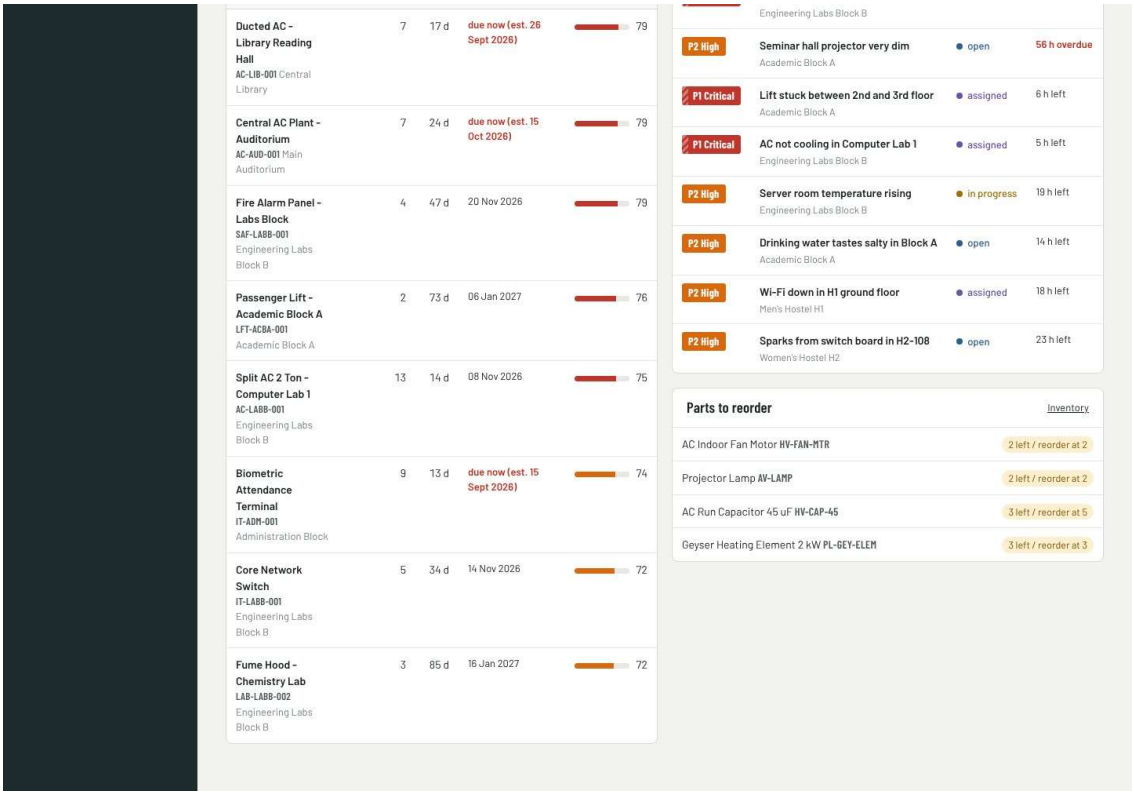


Figure 5. The analytics views and the materialized view `mv_monthly_incident_stats`. (part 2 of 2)

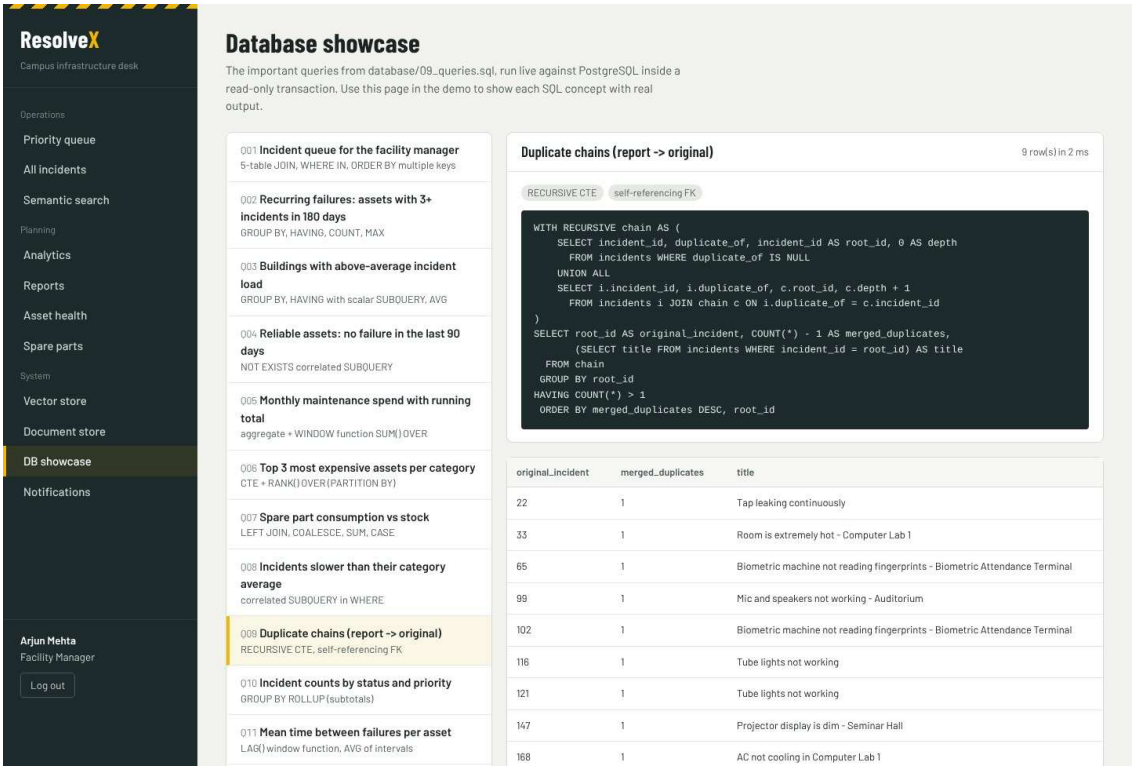


Figure 6. Result of the recursive CTE query (duplicate chains). (part 1 of 2)

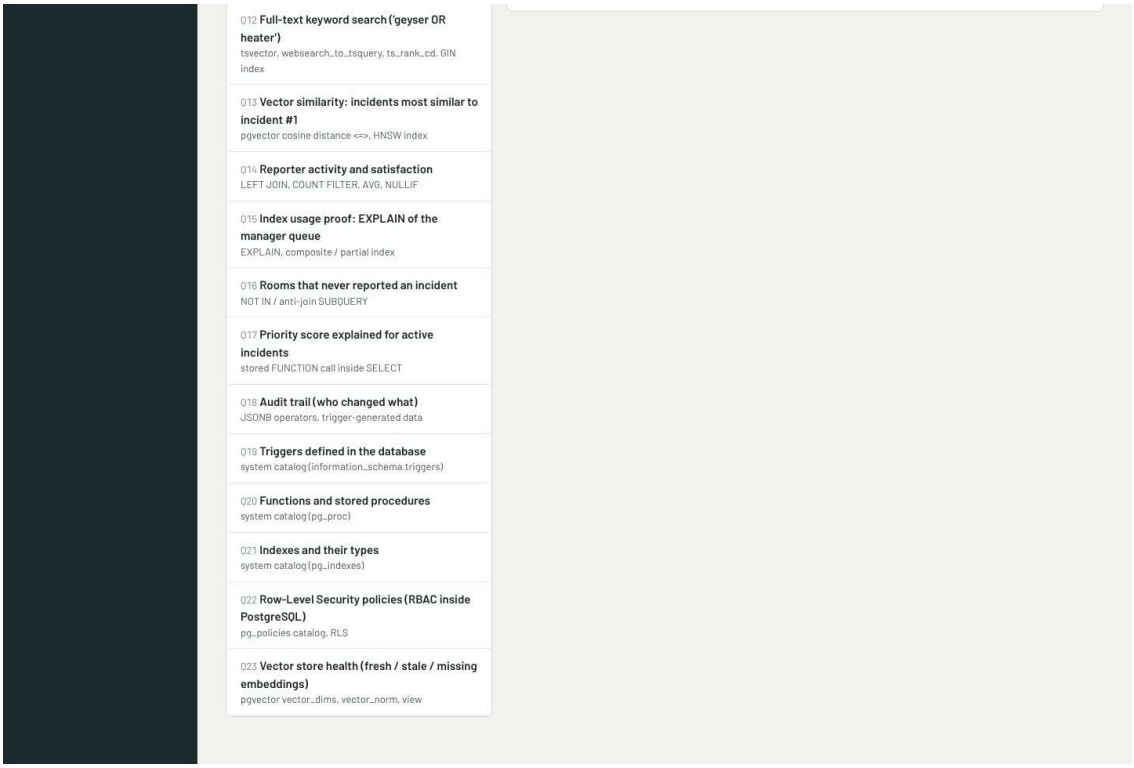


Figure 6. Result of the recursive CTE query (duplicate chains). (part 2 of 2)

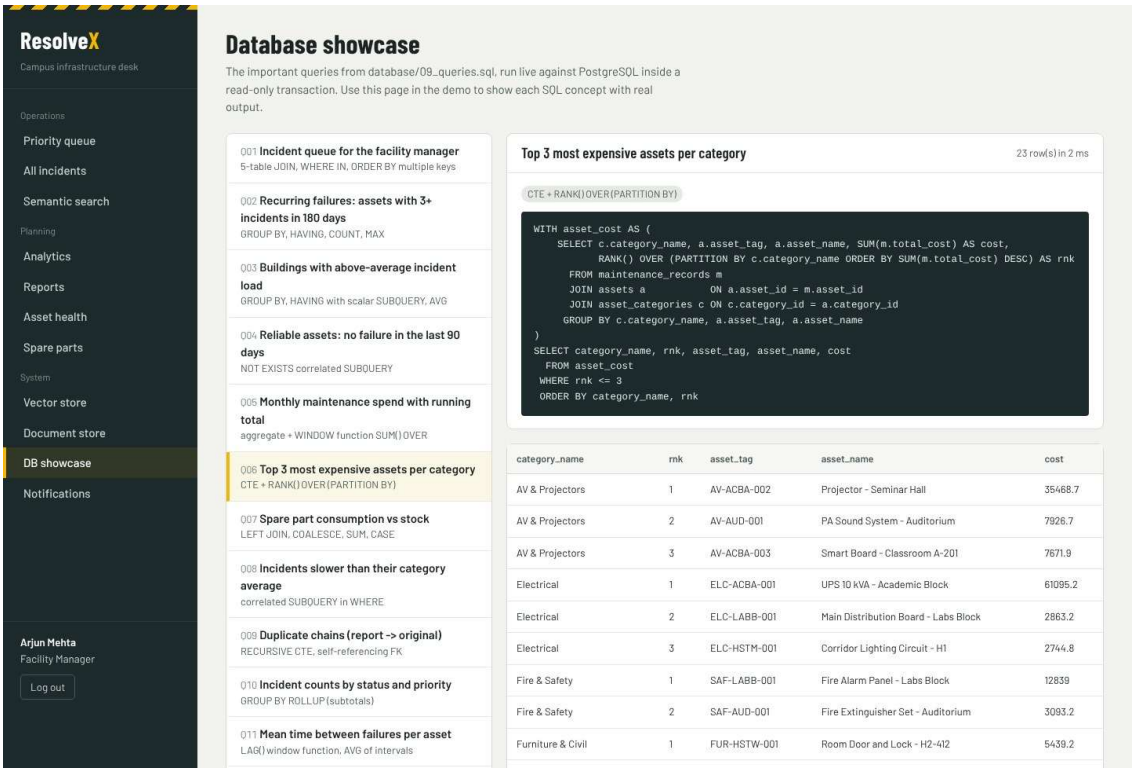


Figure 7. Result of the window function query (RANK). (part 1 of 2)

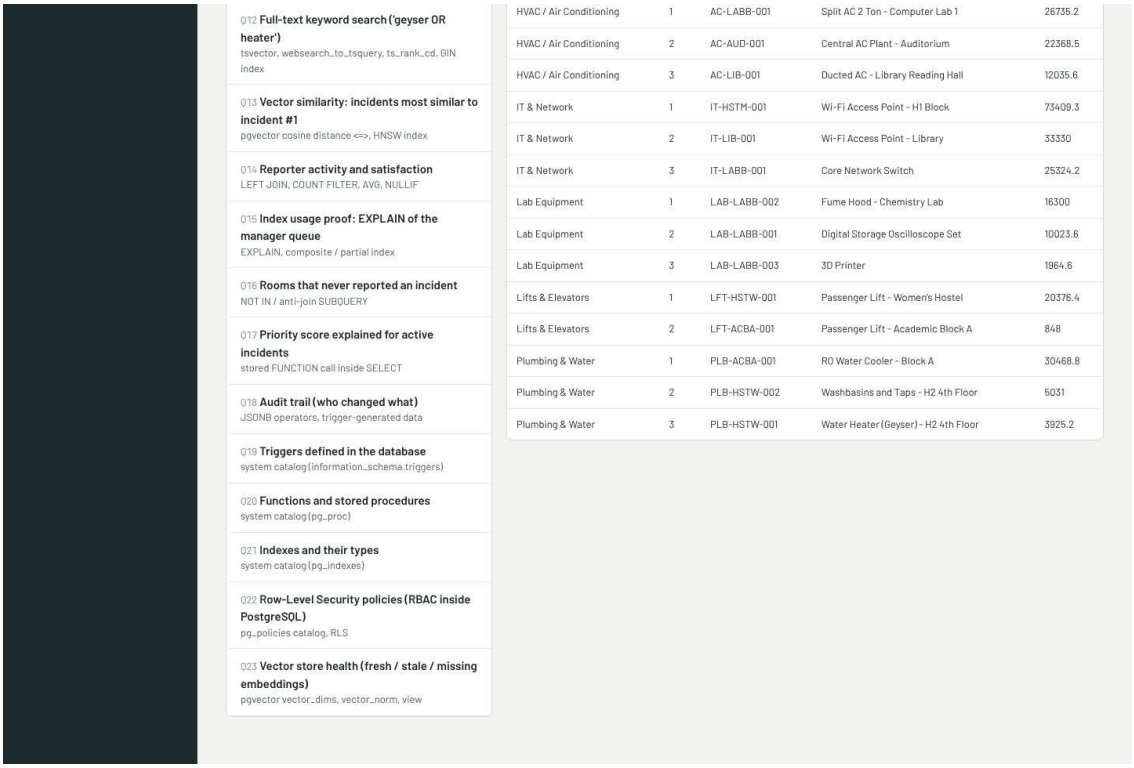


Figure 7. Result of the window function query (RANK). (part 2 of 2)

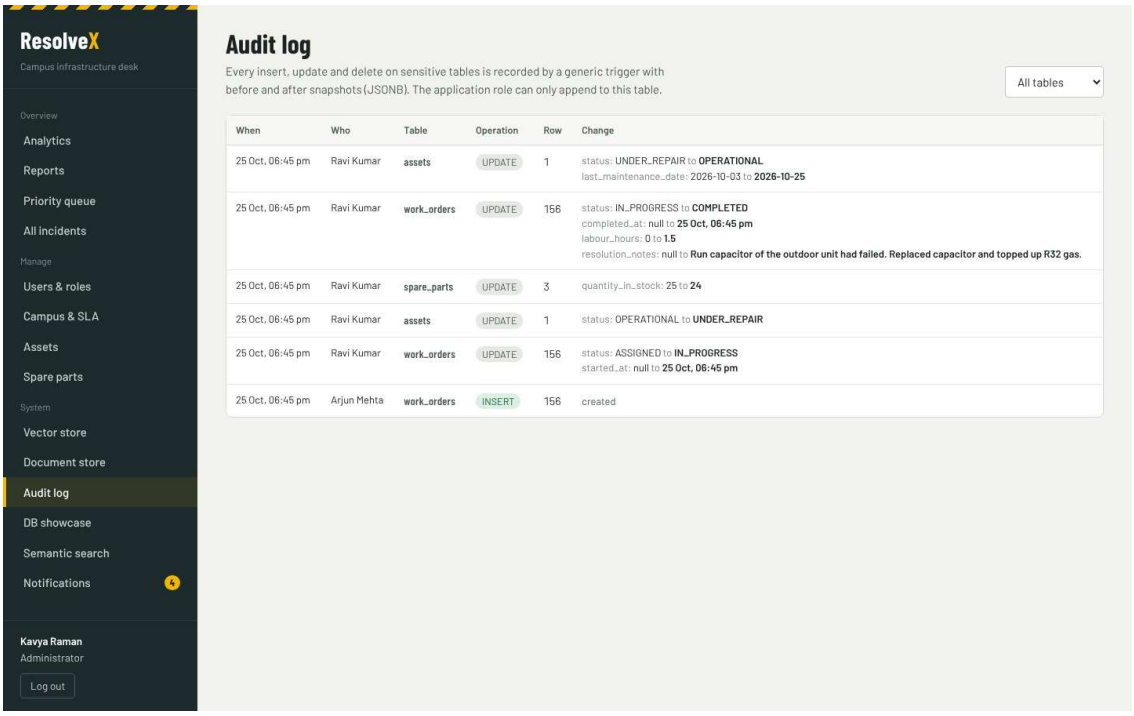


Figure 8. The `audit_log` table, filled by the audit trigger.

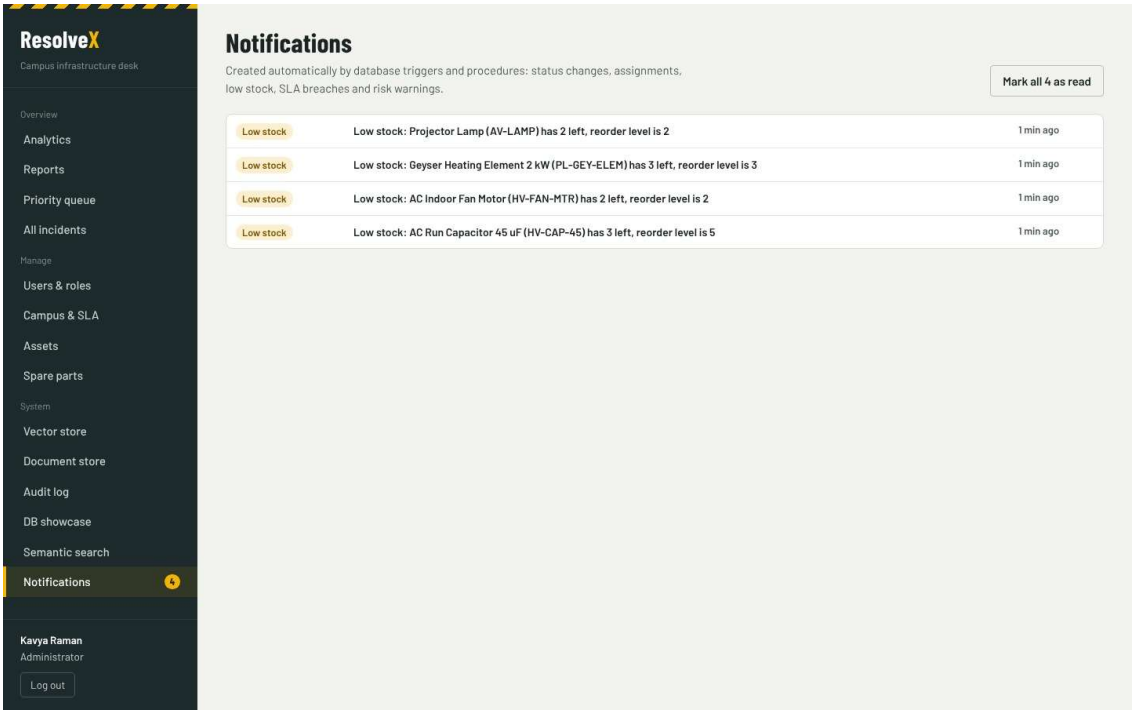


Figure 9. The `alerts` table, filled automatically by triggers.

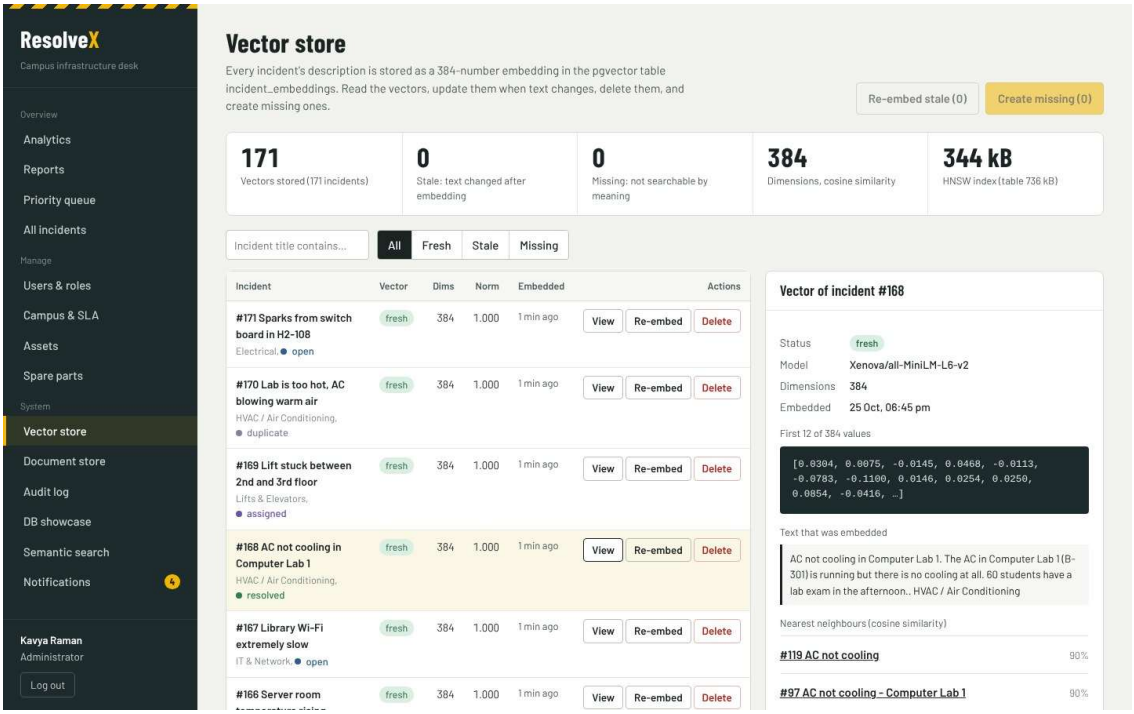


Figure 10. The `incident_embeddings` table: 171 vectors of 384 dimensions with an HNSW index.

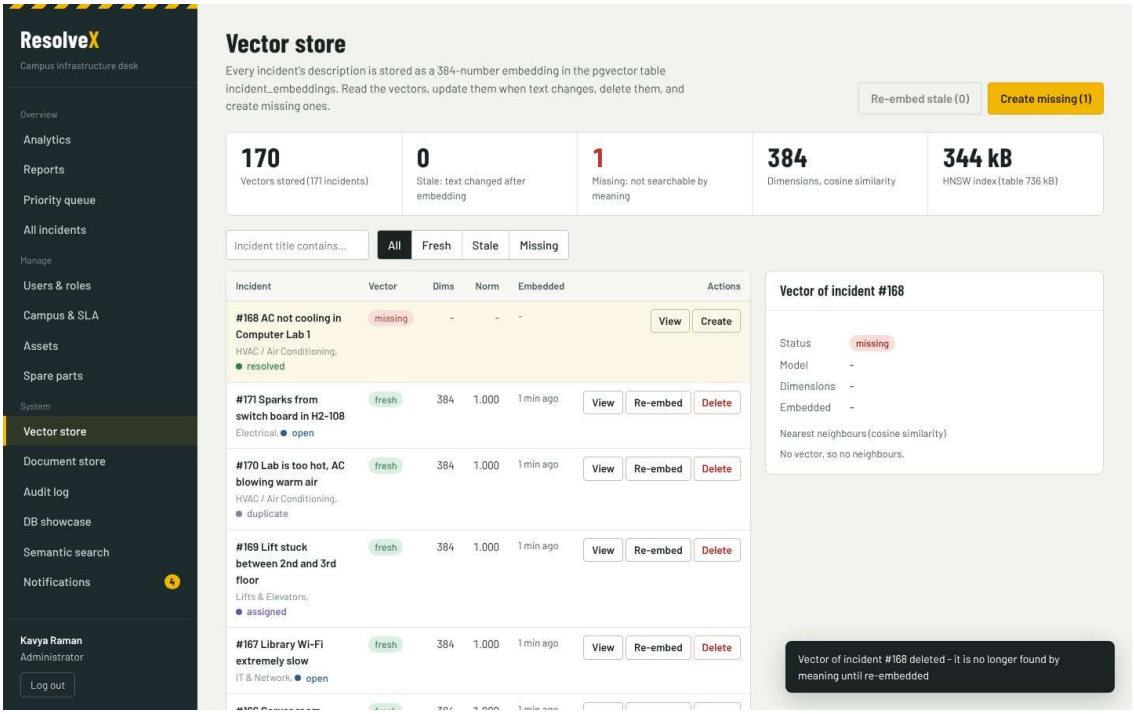


Figure 11. The `incident_embeddings` table after deleting a vector: the incident shows as "missing".

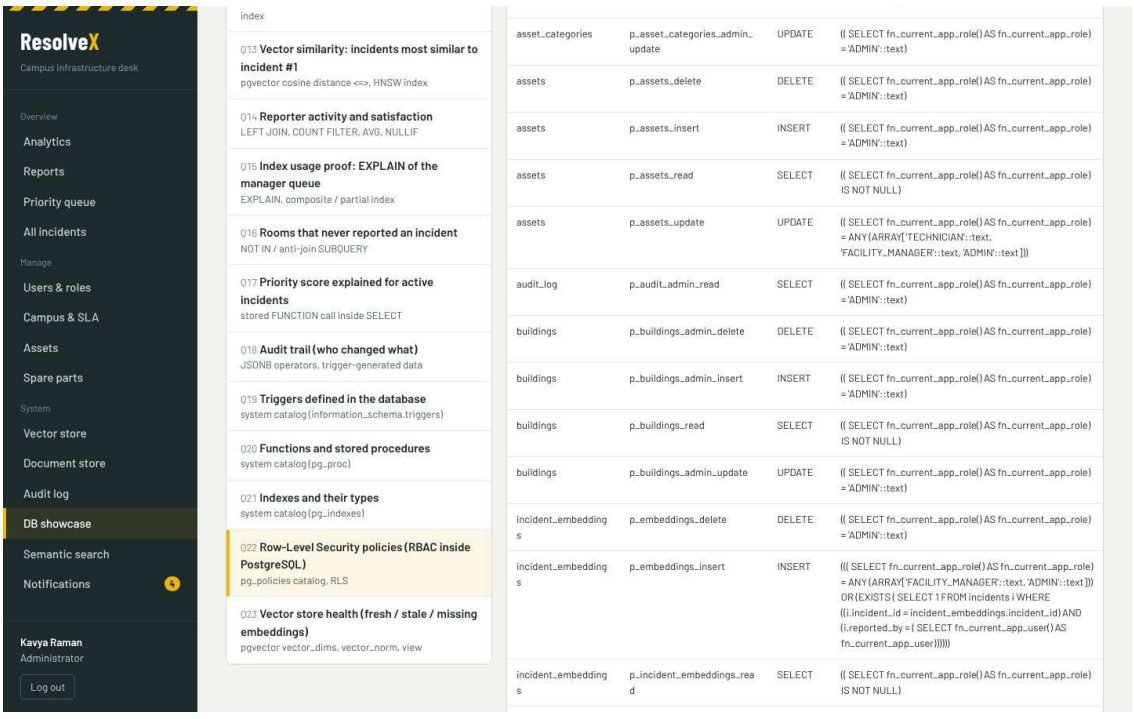


Figure 12. Row-Level Security policies, read from `pg_policies`.

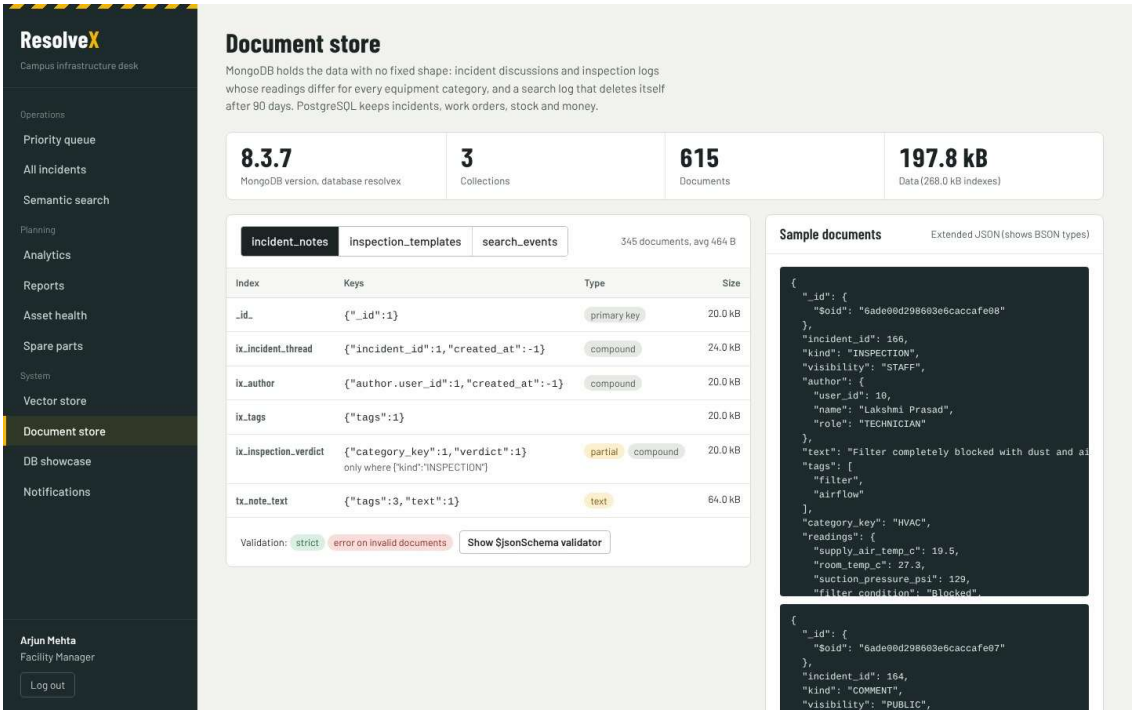


Figure 13. MongoDB collections with document counts and indexes.

9. Next.js – Project Tree Structure

We built the front end and the API using the Next.js 15 App Router. The pages are inside `app/(app)/`, the API routes are inside `app/api/`, common server code is in `lib/`, and all the database scripts are in `database/`. Generated or secret folders and files such as `node_modules/`, `.next/`, `.model-cache/`, `models/` and `.env.local` are listed in `.gitignore`, so they are not pushed to GitHub.

```
resolvex/
├── app/                                Next.js App Router
│   ├── layout.js                       root layout (fonts, global CSS)
│   ├── page.js                         redirects to the user's home page
│   ├── globals.css
│   ├── not-found.js
│   ├── login/page.js                   login screen
│   ├── (app)/                          pages behind login (18 pages)
│   │   ├── layout.js                   sidebar shell, role-based menu
│   │   ├── my/page.js                  student: my reports
│   │   ├── report/page.js              report a problem + live duplicate detection
│   │   ├── search/page.js              semantic / hybrid / keyword search
│   │   ├── incidents/page.js           all incidents
│   │   ├── incidents/[id]/page.js      incident detail, priority breakdown, notes
│   │   ├── queue/page.js               manager priority queue
│   │   ├── work-orders/page.js         technician jobs, complete with parts
│   │   ├── inventory/page.js           spare parts, restock
│   │   ├── assets/page.js              asset health, risk score, MTBF
│   │   ├── analytics/page.js           dashboards from views
│   │   ├── reports/page.js             8 SQL + 7 MongoDB reports, CSV
│   │   ├── showcase/page.js            23 live SQL queries
│   │   ├── vectors/page.js             vector store (read, re-embed, delete)
│   │   ├── nosql/page.js               MongoDB document store
│   │   ├── alerts/page.js              notifications
│   │   └── admin/
│   │       ├── users/page.js           users and roles
│   │       ├── campus/page.js          buildings, rooms, SLA policy
│   │       └── audit/page.js           audit log
│   └── api/                            32 API route handlers
│       ├── auth/login | logout | me/route.js
│       ├── incidents/route.js          POST: incident + embedding in one transaction
│       ├── incidents/similar/route.js  duplicate detection while typing
│       ├── incidents/[id]/route.js     edit + re-embed, status actions
│       ├── incidents/[id]/assign | confirm | duplicate/route.js
│       ├── incidents/[id]/notes/route.js MongoDB notes: find, insertOne
│       ├── notes/[noteId]/route.js     MongoDB notes: edit, vote, delete
│       ├── work-orders/route.js
│       ├── work-orders/[id]/route.js   start, hold, CALL sp_complete_work_order
│       ├── vectors/route.js            list, re-embed stale, create missing
│       ├── vectors/[id]/route.js       DELETE vector (admin)
│       ├── search/route.js             fn_hybrid_search
│       ├── parts/route.js, parts/[id]/route.js, parts/[id]/restock/route.js
│       ├── assets/, maintenance/, analytics/, reports/, showcase/
│       ├── nosql/route.js              document store page data
│       ├── users/, campus/, audit/, alerts/, meta/
├── components/                         React UI pieces
│   ├── Shell.js                        sidebar and layout
│   ├── IncidentTable.js
│   ├── NotesPanel.js                   comments + inspection log form
│   └── ui.js                           buttons, tags, status badges
├── lib/                                server-side helpers
│   ├── db.js                           PostgreSQL pool, transactions, set_config for RLS
│   ├── auth.js                         JWT session, requireRole()
│   ├── api.js                          input validation, error handling
│   ├── embeddings.js                   all-MiniLM-L6-v2 embedding model
│   ├── incidents.js                    embed missing incidents
│   ├── mongo.js                        MongoDB connection
│   ├── notes.js                        note access rules, template cleaning
│   ├── reports.js                      8 SQL reports
│   ├── mongo-reports.js                7 aggregation pipelines
│   ├── showcase.js                     loads 09_queries.sql
│   └── context.js
├── middleware.js                       page protection by role
├── database/
│   ├── 00_drop.sql … 10_viva_live_demo.sql   (11 SQL scripts)
│   └── mongodb/
│       ├── schema.mjs                  templates, validators, indexes
│       └── live_demo.mongosh.js        MongoDB demo commands
├── scripts/
│   ├── setup-db.mjs                    npm run db:setup (PostgreSQL + MongoDB + vectors)
│   ├── setup-mongo.mjs                 npm run db:mongo
```

```
│   ├── embed-incidents.mjs              npm run db:embed
│   ├── generate seed.py, generate data dictionary.py, take screenshots.py
├── tests/
│   ├── run-db-tests.mjs                 53 SQL tests
│   ├── run-nosql-tests.mjs              27 MongoDB tests
│   └── run-api-tests.mjs                41 API and security tests
├── docs/                                design docs, diagrams, screenshots, test results
├── public/icon.svg
├── docker-compose.yml                   PostgreSQL+pgvector and MongoDB
├── .env.example                         settings template (no real secrets)
├── next.config.mjs, jsconfig.json
├── package.json, package-lock.json
└── README.md
```

Commands to run the project

```
docker compose up -d                  # start PostgreSQL + pgvector and MongoDB
npm install                           # install packages
npm run db:setup                      # build both databases, load data, create embeddings
npm run dev                           # http://localhost:3000
npm run test:db                       # 53 SQL tests
npm run test:nosql                    # 27 MongoDB tests
npm run test:api                      # 41 API and security tests
```

10. Sample Screenshots of Application Code and Demo

The code screenshots below are taken from our repository, and the demo screenshots were taken from the app while it was running with our sample data. For the demo we used four accounts, student@, tech@, manager@ and admin@resolvex.edu, all with the password `Resolve@123`.

10.1 Application code

lib/db.js lines 31–55

```
 *   BEGIN -> set_config('app.current_user_id', id, true) -> fn -> COMMIT
 * The setting is read by Row-Level Security policies (who may see / change
 * which rows) and by the audit and history triggers (who made a change).
 * Any error -> ROLLBACK (all-or-nothing).
 */
async function inUserTransaction(userId, fn) {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    await client.query("SELECT set_config('app.current_user_id', $1, true)", [String(userId ?? "")]);
    const result = await fn(client);
    await client.query("COMMIT");
    return result;
  } catch (err) {
    await client.query("ROLLBACK").catch(() => {});
    throw err;
  } finally {
    client.release();
  }
}

/**
 * Parameterised query. Inside an API request it runs as the logged-in user
 * (so RLS applies); without a session (e.g. before login) it runs with no
 * user, and RLS returns no rows from protected tables.
```

Figure 14. `lib/db.js`: each request runs inside a transaction that first tells PostgreSQL which user is logged in (needed for RLS). If anything fails, it rolls back.

lib/auth.js lines 78–87

```
/** Throws 401/403 unless the user is logged in with one of the roles. */
export async function requireRole(...roles) {
  const session = await getSession();
  if (!session) throw new HttpError(401, "Please log in");
  if (roles.length && !roles.includes(session.role)) {
    throw new HttpError(403, "Your role is not allowed to do this");
  }
  setCurrentSession(session); // every later SQL statement in this request runs as this user (RLS)
  return session;
}
```

Figure 15. `lib/auth.js`: `requireRole()`, which we call at the start of every API route to check the login and the role.

app/api/work-orders/[id]/route.js lines 9–56

```
export const POST = route(async (req, { params }) => {
  const s = await requireRole("TECHNICIAN", "FACILITY_MANAGER", "ADMIN");
  const id = v.int((await params).id, "Work order id");
  const b = await body(req);
  const wo = await queryOne("SELECT work_order_id, technician_id, incident_id, status FROM work_orders WHERE work_order_id = $1", [id]);
  if (!wo) return fail("Work order not found", 404);

  const owner = s.role === "TECHNICIAN" && wo.technician_id === s.id;
  const manager = ["FACILITY_MANAGER", "ADMIN"].includes(s.role);
  if (b.action !== "cancel" && !owner) throw new HttpError(403, "This work order is not assigned to you");
  if (b.action === "cancel" && !manager) throw new HttpError(403, "Only managers can cancel work orders");

  const message = await withTransaction(s.id, async (c) => {
    switch (b.action) {
      case "start": {
        const r = await c.query(
          "UPDATE work_orders SET status = 'IN_PROGRESS', started_at = now() WHERE work_order_id = $1 AND status = 'ASSIGNED'",
          [id]
        );
        if (!r.rowCount) throw new HttpError(409, "Work order is not in ASSIGNED state");
        return "Work started - incident and asset status updated by trigger";
      }
      case "hold":
        await c.query("UPDATE incidents SET status = 'ON_HOLD' WHERE incident_id = $1", [wo.incident_id]);
        return "Incident put on hold";
      case "resume":
        await c.query("UPDATE incidents SET status = 'IN_PROGRESS' WHERE incident_id = $1", [wo.incident_id]);
        return "Work resumed";
      case "complete": {
        const hours = v.num(b.labour_hours, "Labour hours", { min: 0.1, max: 200 });
        const notes = v.str(b.notes, "Resolution notes", { min: 10, max: 2000 });
        const parts = Array.isArray(b.parts) ? b.parts : [];
        const clean = parts.map((p) => ({
          part_id: v.int(p.part_id, "Part"),
          quantity: v.int(p.quantity, "Quantity", { min: 1, max: 1000 }),
        }));
        await c.query("CALL sp_complete_work_order($1, $2, $3, $4, $5::jsonb)", [id, s.id, hours, notes, JSON.stringify(clean)]);
        return "Work order completed - stock deducted, cost recorded, incident resolved";
      }
      case "cancel":
        await c.query("UPDATE work_orders SET status = 'CANCELLED' WHERE work_order_id = $1 AND status IN ('ASSIGNED','IN_PROGRESS')", [id]);
        return "Work order cancelled - incident returned to the queue";
      default:
        throw new HttpError(400, "Unknown action");
    }
  });
  return ok({ message });
});
```

Figure 16. `app/api/work-orders/[id]/route.js`: the actions a technician can take on a job. "Complete" calls the `sp_complete_work_order` procedure.

lib/embeddings.js lines 8–46

```
import { pipeline, env } from "@huggingface/transformers";
import path from "node:path";

export const EMBEDDING_MODEL = process.env.EMBEDDING_MODEL || "Xenova/all-MiniLM-L6-v2";
export const EMBEDDING_DIM = 384;

// Look for a local copy first (./models/Xenova/all-MiniLM-L6-v2), otherwise
// download once from the Hugging Face hub and cache it in ./.model-cache
env.localModelPath = process.env.EMBEDDING_MODEL_PATH || path.join(process.cwd(), "models");
env.cacheDir = path.join(process.cwd(), ".model-cache");
env.allowLocalModels = true;
env.allowRemoteModels = process.env.EMBEDDING_ALLOW_REMOTE !== "false";

let extractorPromise = globalThis.__resolvexExtractor;

function getExtractor() {
  if (!extractorPromise) {
    extractorPromise = pipeline("feature-extraction", EMBEDDING_MODEL, { dtype: "fp32" });
    globalThis.__resolvexExtractor = extractorPromise; // survive Next.js hot reloads
  }
  return extractorPromise;
}

/** Text that represents an incident for semantic search. */
export function incidentText({ title, description, category }) {
  return [title, description, category].filter(Boolean).join(". ");
}

/** Returns a plain JS array of 384 numbers. */
export async function embed(text) {
  const extractor = await getExtractor();
  const output = await extractor(text, { pooling: "mean", normalize: true });
  return Array.from(output.data);
}

/** pgvector literal: '[0.1,0.2,...]' */
export function toPgVector(vec) {
  return "[" + vec.map((v) => Number(v).toFixed(6)).join(",") + "]";
}
```

Figure 17. `lib/embeddings.js`: loads the all-MiniLM-L6-v2 model and converts text into a 384-number vector.

app/api/vectors/[id]/route.js lines 1–18

```
// DELETE /api/vectors/:id   VECTOR DELETE (Admin only)
// Removes an incident's embedding from the vector store. The incident row stays,
// but it no longer appears in semantic search or duplicate detection until it is
// re-embedded. RLS policy p_embeddings_delete allows this only for ADMIN.
import { ok, route, v } from "@/lib/api";
import { requireRole } from "@/lib/auth";
import { withTransaction } from "@/lib/db";

export const DELETE = route(async (req, { params }) => {
  const s = await requireRole("ADMIN");
  const id = v.int((await params).id, "Incident id");
  const r = await withTransaction(s.id, (c) => c.query("DELETE FROM incident_embeddings WHERE incident_id = $1", [id]));
  return ok({
    message: r.rowCount
      ? `Vector of incident #${id} deleted - it is no longer found by meaning until re-embedded`
      : `Incident #${id} had no vector`,
  });
});
```

Figure 18. `app/api/vectors/[id]/route.js`: deleting a vector (only the admin can do this).

app/api/incidents/[id]/notes/route.js lines 26–62

```
export const POST = route(async (req, { params }) => {
  const s = await requireRole();
  const id = v.int((await params).id, "Incident id");
  const access = await incidentAccess(s, id);
  const b = await body(req);
  const kind = v.oneOf(b.kind || "COMMENT", "kind", ["COMMENT", "INSPECTION"]);
  if (kind === "INSPECTION" && !access.canInspect) throw new HttpError(403, "Only the technician assigned to this incident, or a manager, can log an inspection");
  const text = v.str(b.text, kind === "INSPECTION" ? "Findings" : "Comment", { min: 2, max: 2000 });
  const visibility = v.oneOf(b.visibility || (kind === "INSPECTION" ? "STAFF" : "PUBLIC"), "visibility", ["PUBLIC", "STAFF"]);
  if (visibility === "STAFF" && !access.staff) throw new HttpError(403, "Only staff can post internal notes");

  const now = new Date();
  const doc = {
    incident_id: id,
    kind,
    visibility,
    author: { user_id: s.id, name: s.name, role: s.role }, // snapshot: keeps the name as it was when written
    text,
    created_at: now,
    updated_at: now,
  };
  const tags = cleanTags(b.tags);
  if (tags.length) doc.tags = tags;

  if (kind === "INSPECTION") {
    const template = await templateFor(access.incident.category_id);
    if (!template) throw new HttpError(400, "No inspection template for this category");
    doc.category_key = template._id;
    doc.readings = cleanReadings(template, b.readings);
    doc.checklist = cleanChecklist(template, b.checklist);
    doc.verdict = v.oneOf(b.verdict, "Verdict", VERDICTS);
  }

  const res = await (await collection("incident_notes")).insertOne(doc);
  return ok({ message: kind === "INSPECTION" ? "Inspection log saved" : "Comment posted", note: present({ ...doc, _id: res.insertedId }, s) }, 201);
});

```

Figure 19. `app/api/incidents/[id]/notes/route.js`: saving a comment or inspection log in MongoDB, using the template of that category.

middleware.js lines 1–28

```
// ============================================================================
// Route protection (runs before every page / API request).
//  * not logged in  -> pages redirect to /login, APIs return 401
//  * wrong role     -> pages redirect to the user's home page
// API routes additionally check roles themselves (defence in depth).
// ============================================================================
import { NextResponse } from "next/server";
import { jwtVerify } from "jose/jwt/verify"; // only the verifier: keeps the Edge bundle small

const PAGE_ACCESS = [
  ["/work-orders", ["TECHNICIAN"]],
  ["/queue", ["FACILITY_MANAGER", "ADMIN"]],
  ["/incidents", ["FACILITY_MANAGER", "ADMIN", "TECHNICIAN", "STUDENT_STAFF"]],
  ["/analytics", ["FACILITY_MANAGER", "ADMIN"]],
  ["/assets", ["FACILITY_MANAGER", "ADMIN"]],
  ["/inventory", ["TECHNICIAN", "FACILITY_MANAGER", "ADMIN"]],
  ["/showcase", ["FACILITY_MANAGER", "ADMIN"]],
  ["/vectors", ["FACILITY_MANAGER", "ADMIN"]],
  ["/nosql", ["FACILITY_MANAGER", "ADMIN"]],
  ["/reports", ["FACILITY_MANAGER", "ADMIN"]],
  ["/admin", ["ADMIN"]],
];

const HOME = {
  STUDENT_STAFF: "/my",
  TECHNICIAN: "/work-orders",
  FACILITY_MANAGER: "/queue",
  ADMIN: "/analytics",
```

Figure 20. `middleware.js`: decides which pages each role is allowed to open. It runs before every request.

10.2 Demo

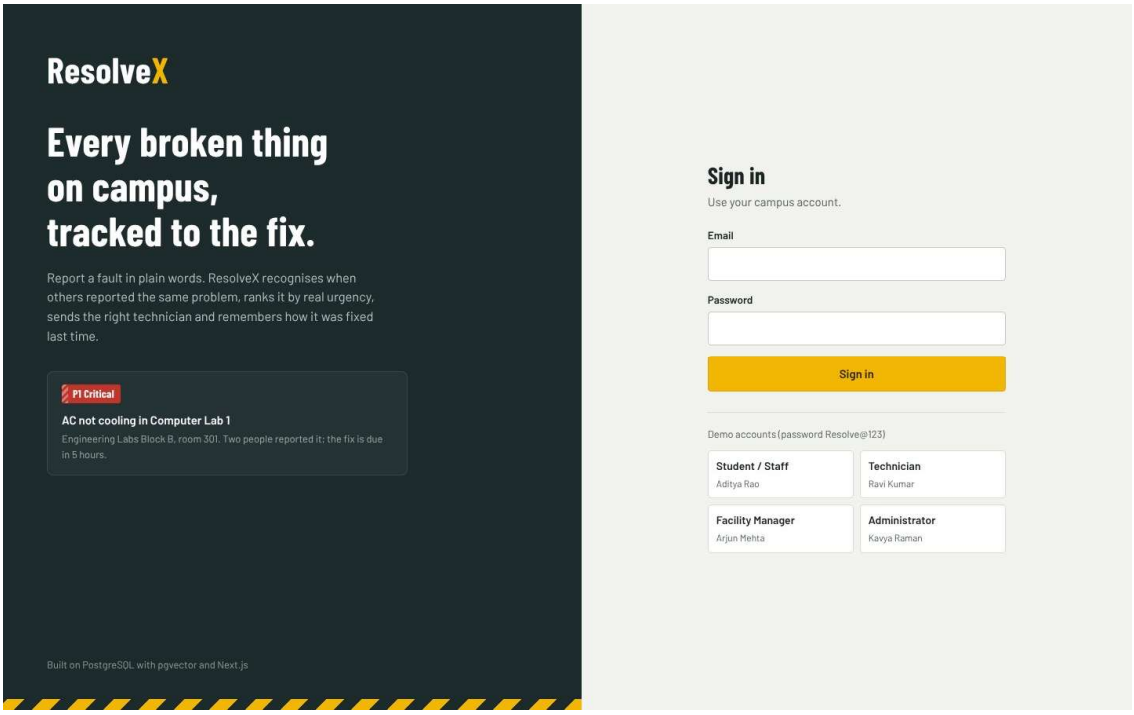


Figure 21. The login page. All four roles use the same login screen.

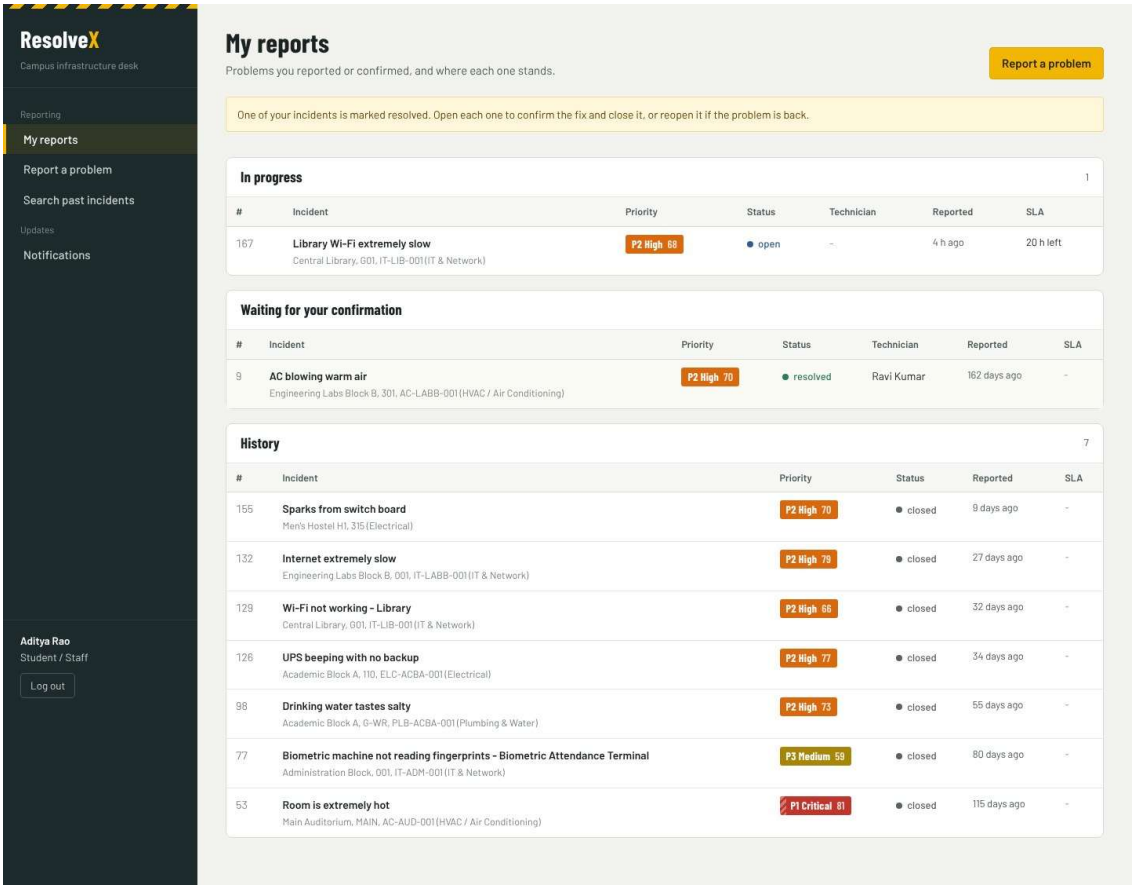


Figure 22. A student's "My reports" page, showing the status, priority and deadline of each report.

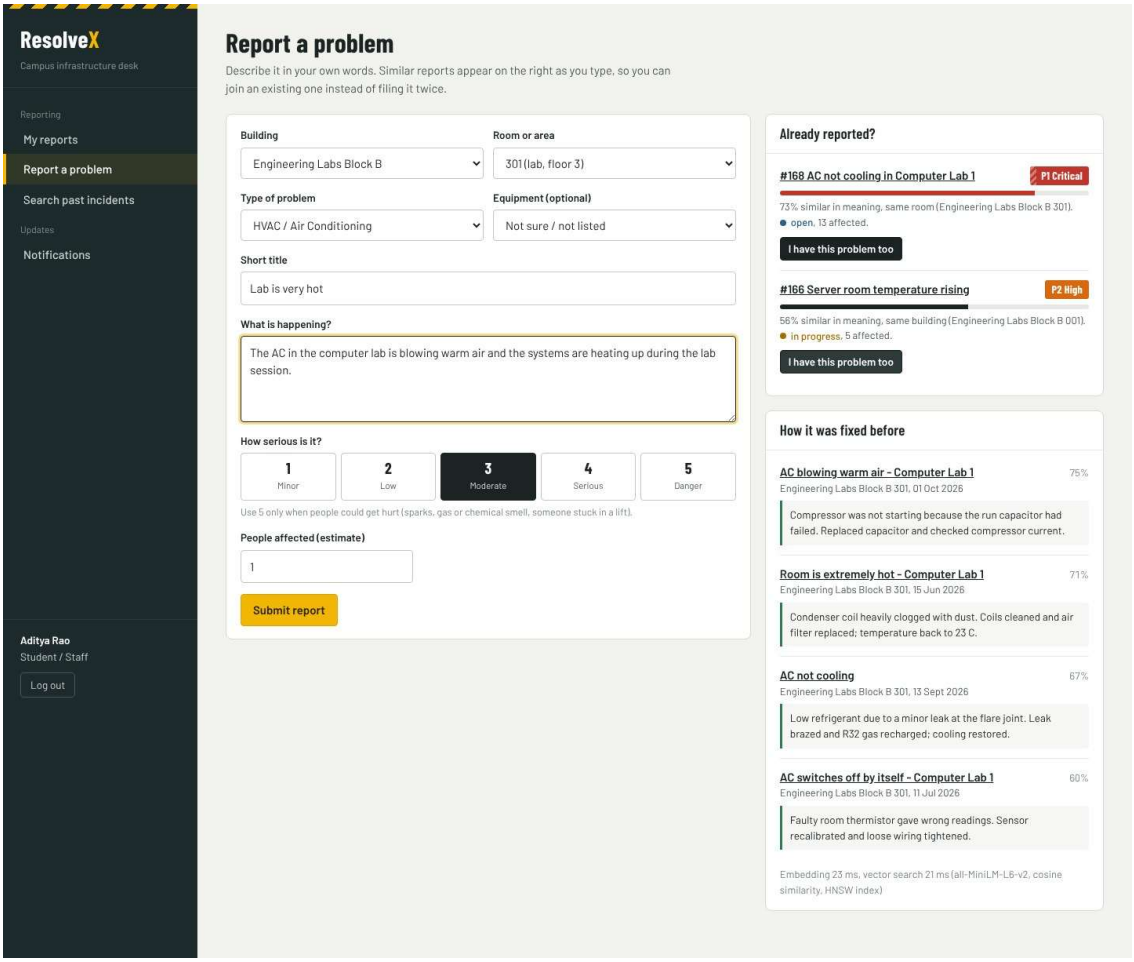


Figure 23. Vector search while reporting. The student typed “Lab is very hot” and the app found #168 “AC not cooling” (73% similar in meaning), along with how similar problems were fixed earlier.

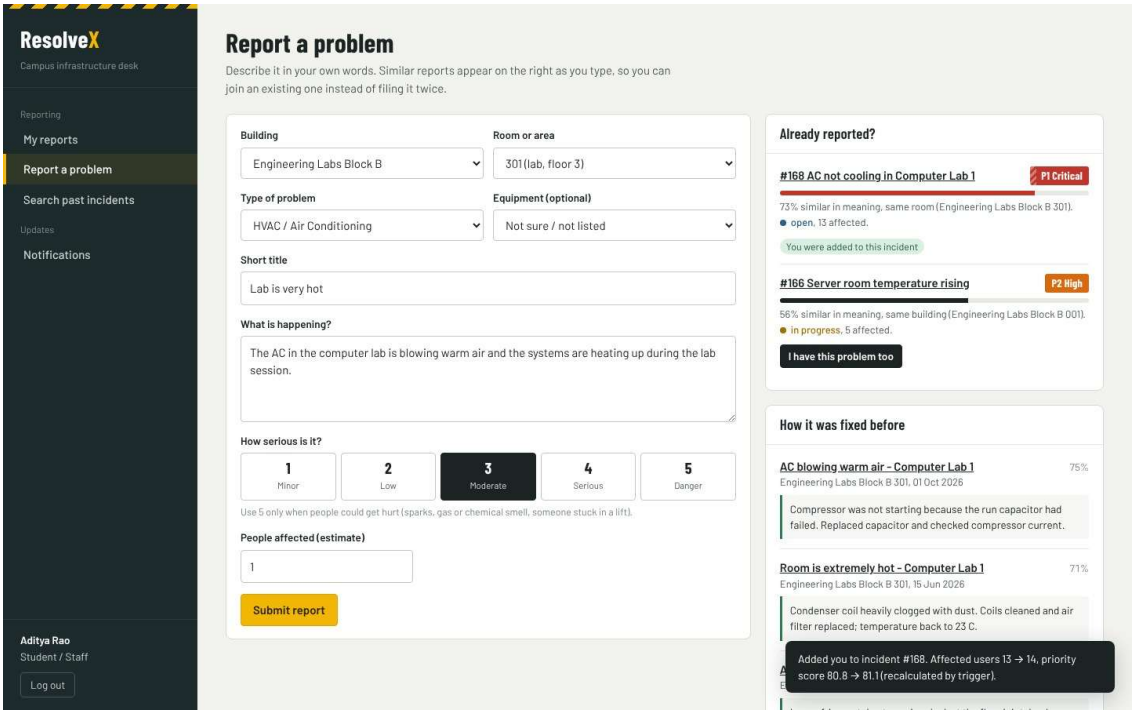


Figure 24. After clicking “I have this problem too”, a trigger increases the affected count and the priority score is recalculated.

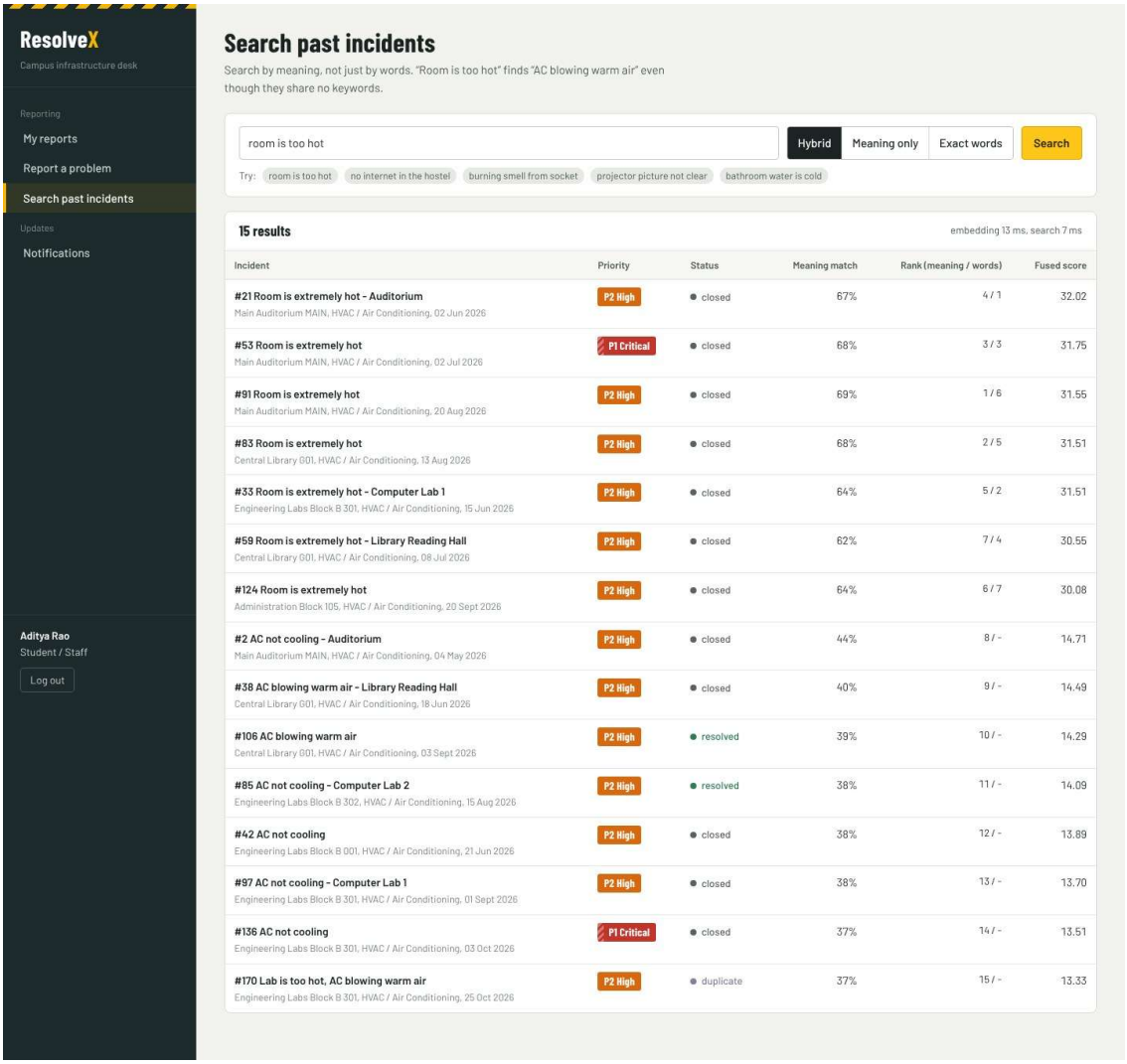


Figure 25. The search page, which has Hybrid, Meaning only and Exact words modes.

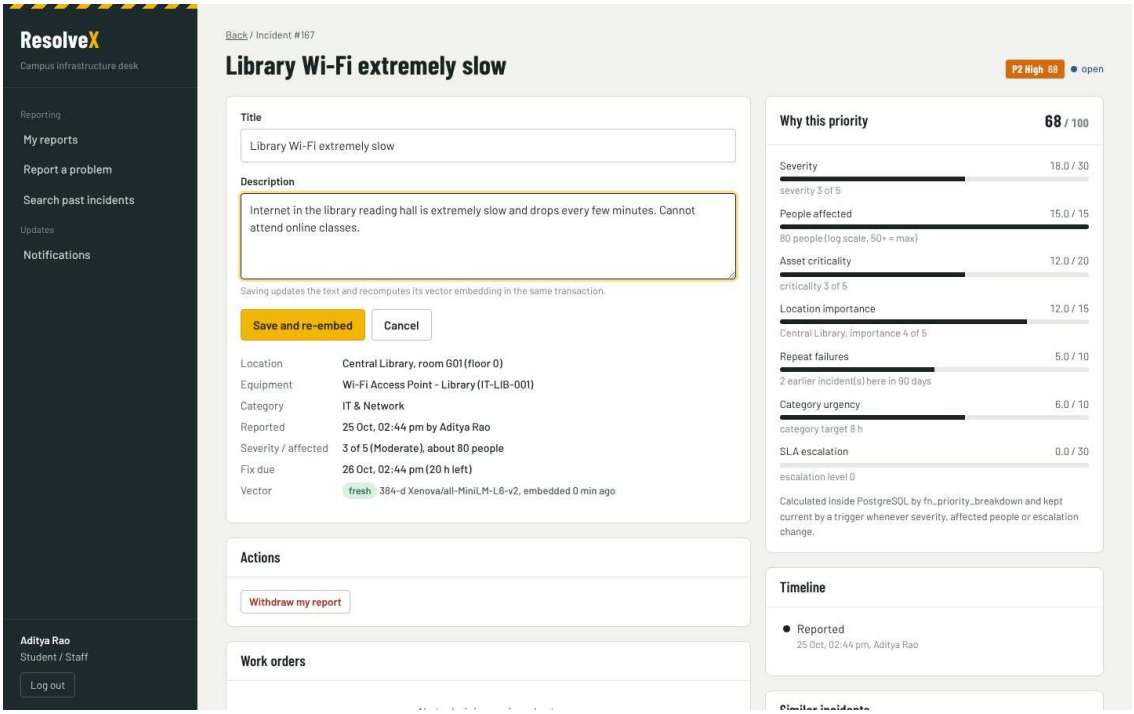


Figure 26. Editing a report's title and description (vector update).

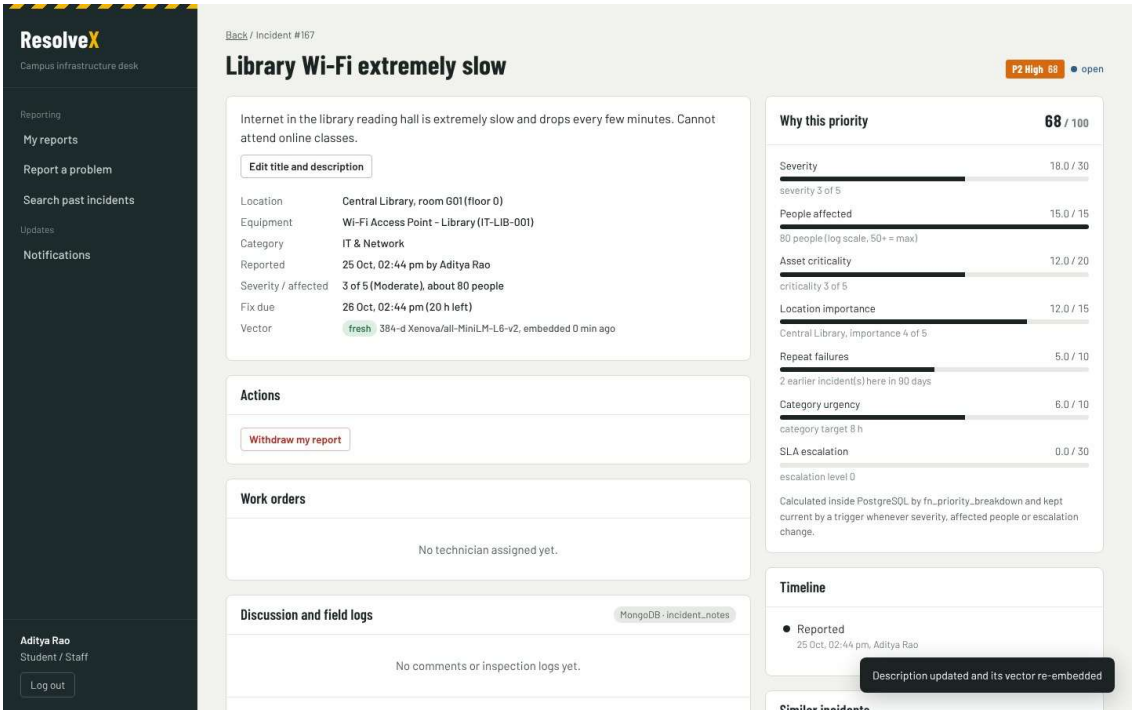


Figure 27. After saving the edit, the vector is recalculated and shows as "fresh".

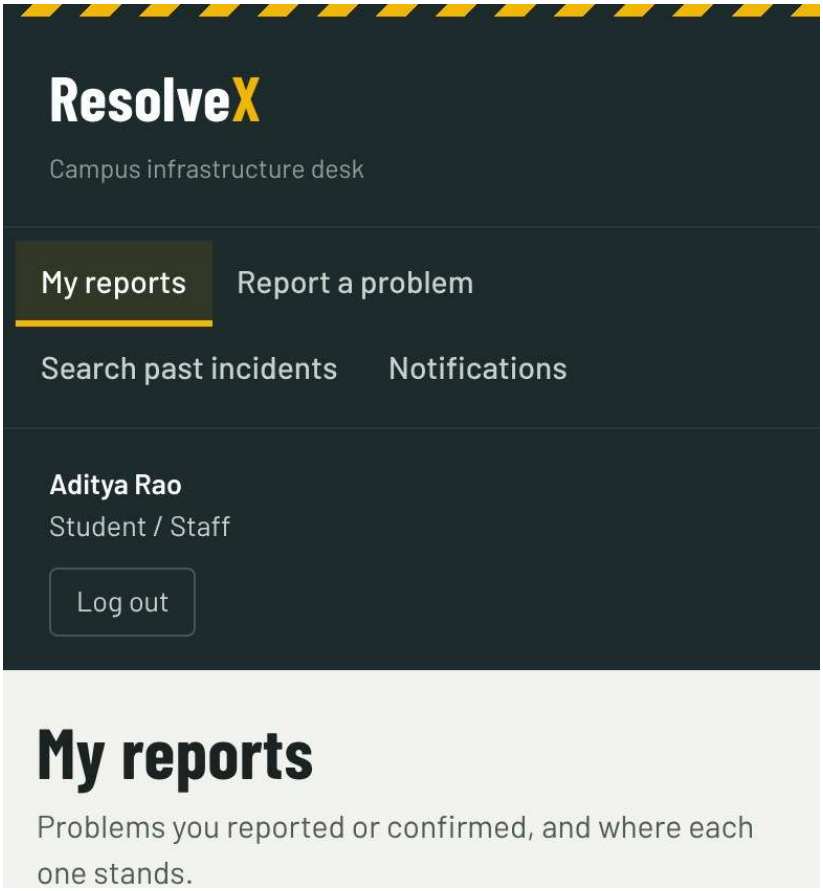


Figure 28. The student pages also work on a phone screen. (part 1 of 2)

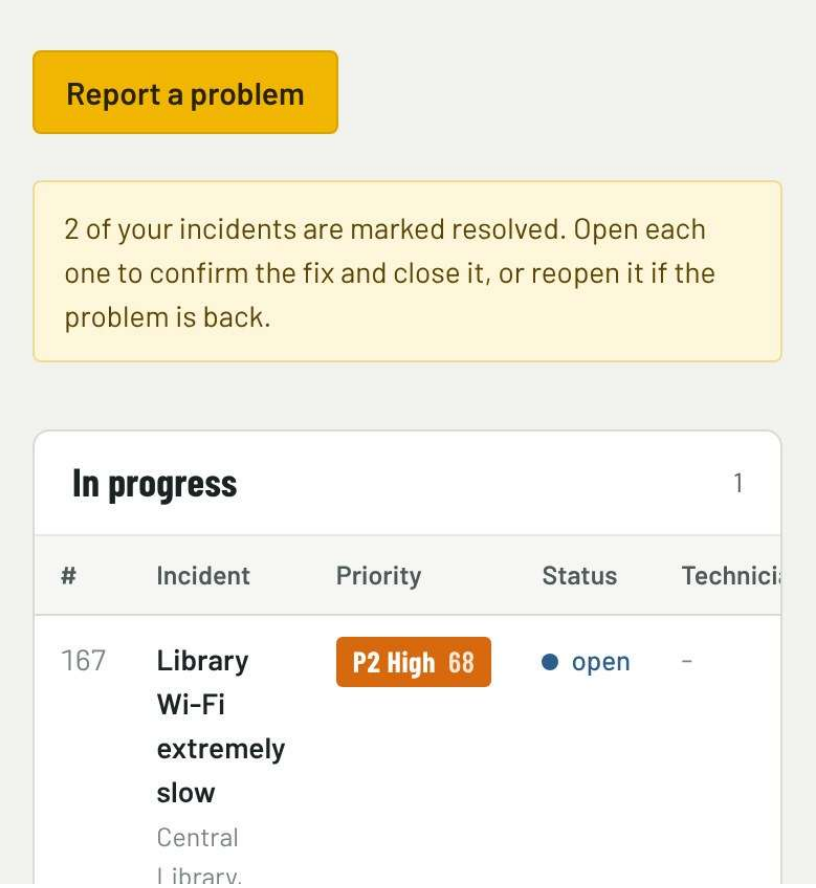


Figure 28. The student pages also work on a phone screen. (part 2 of 2)

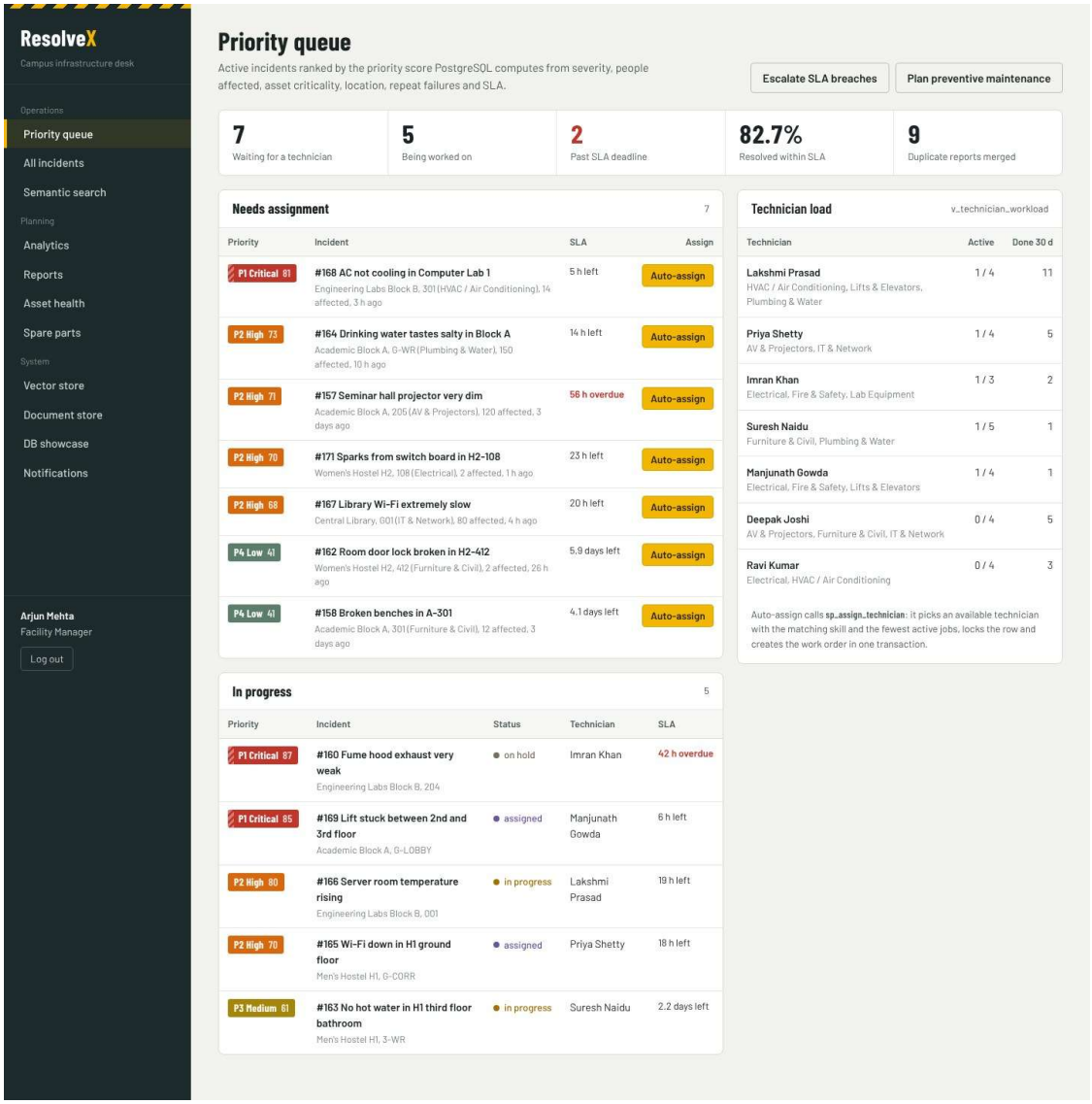


Figure 29. The manager's priority queue, sorted by the score that PostgreSQL calculated.

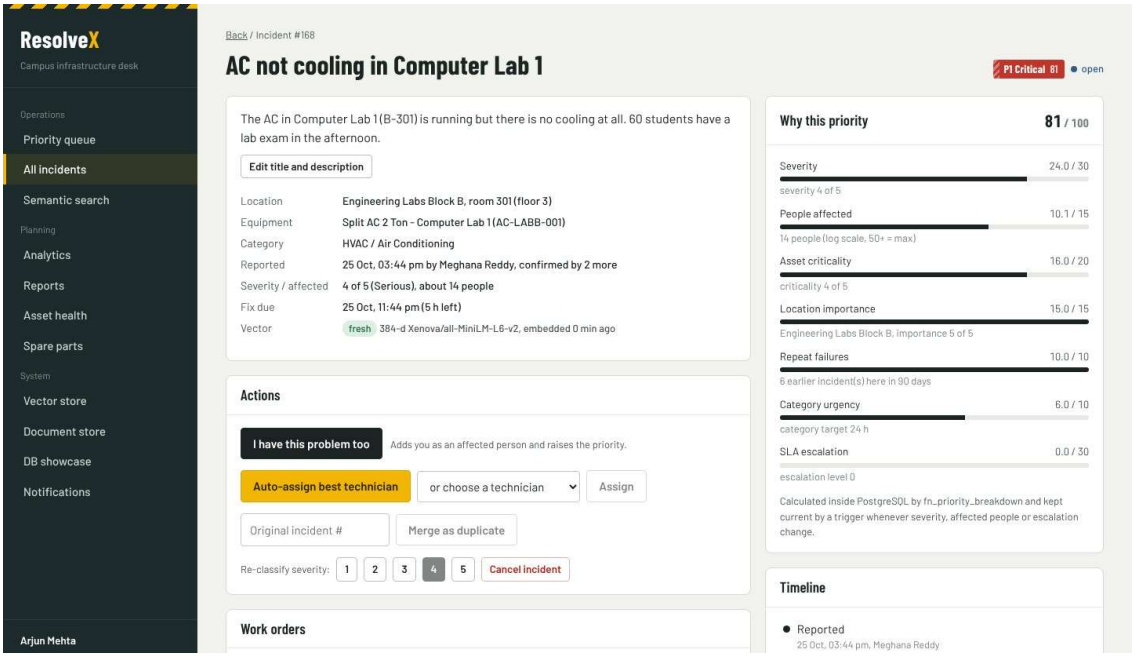


Figure 30. An incident's details page. The "Why this priority" panel comes from `fn_priority_breakdown`. (part 1 of 2)

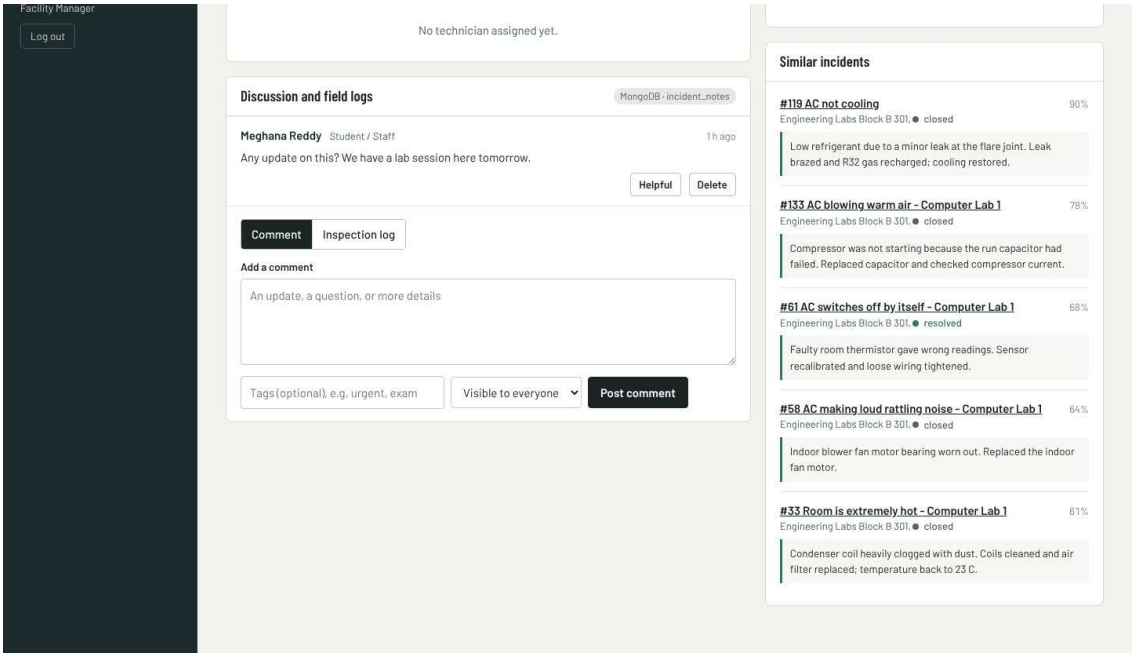


Figure 30. An incident's details page. The "Why this priority" panel comes from `fn_priority_breakdown`. (part 2 of 2)

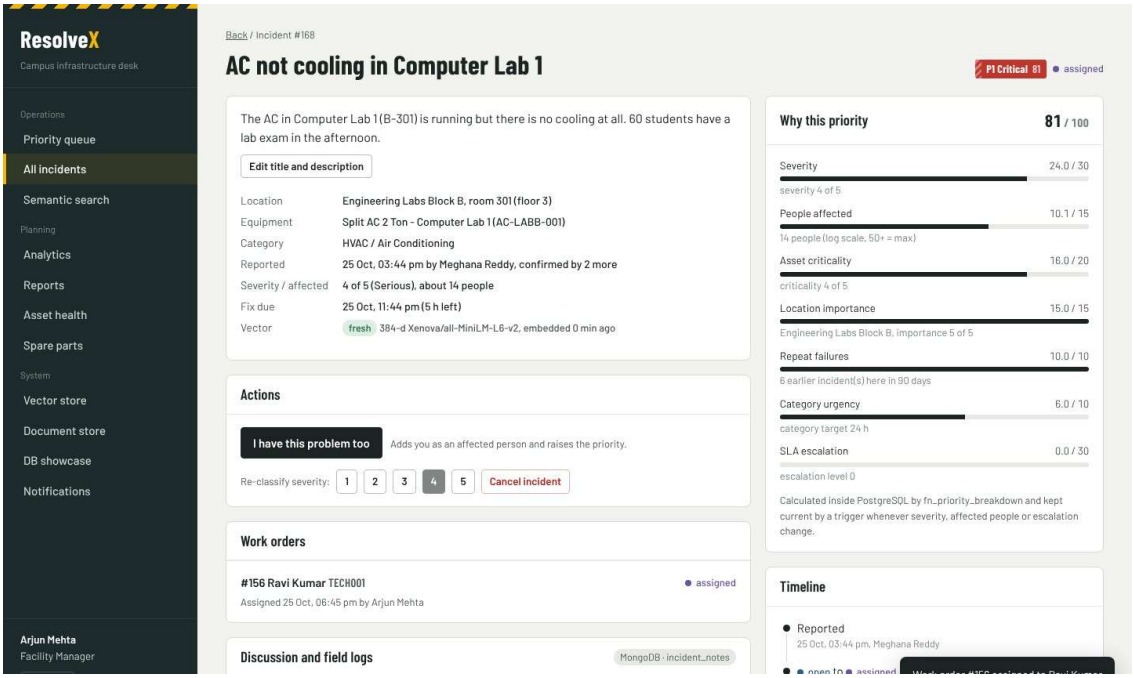


Figure 31. After Auto-assign, `sp_assign_technician` has created a work order for the least busy technician with the right skill. (part 1 of 2)

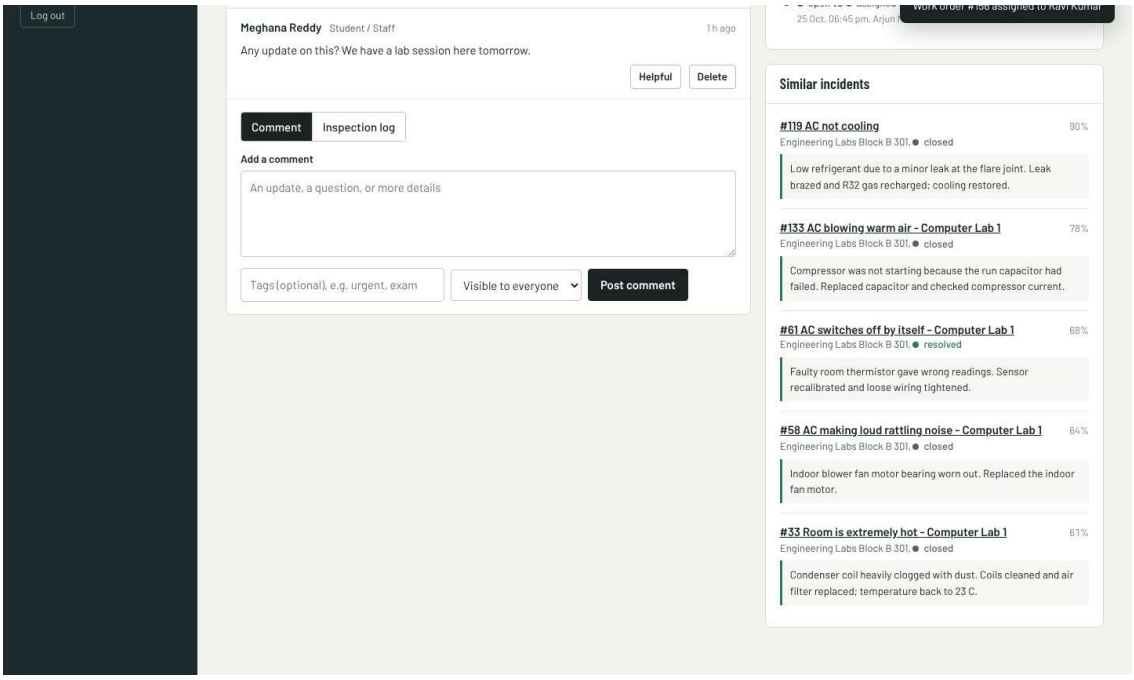


Figure 31. After Auto-assign, `sp_assign_technician` has created a work order for the least busy technician with the right skill. (part 2 of 2)

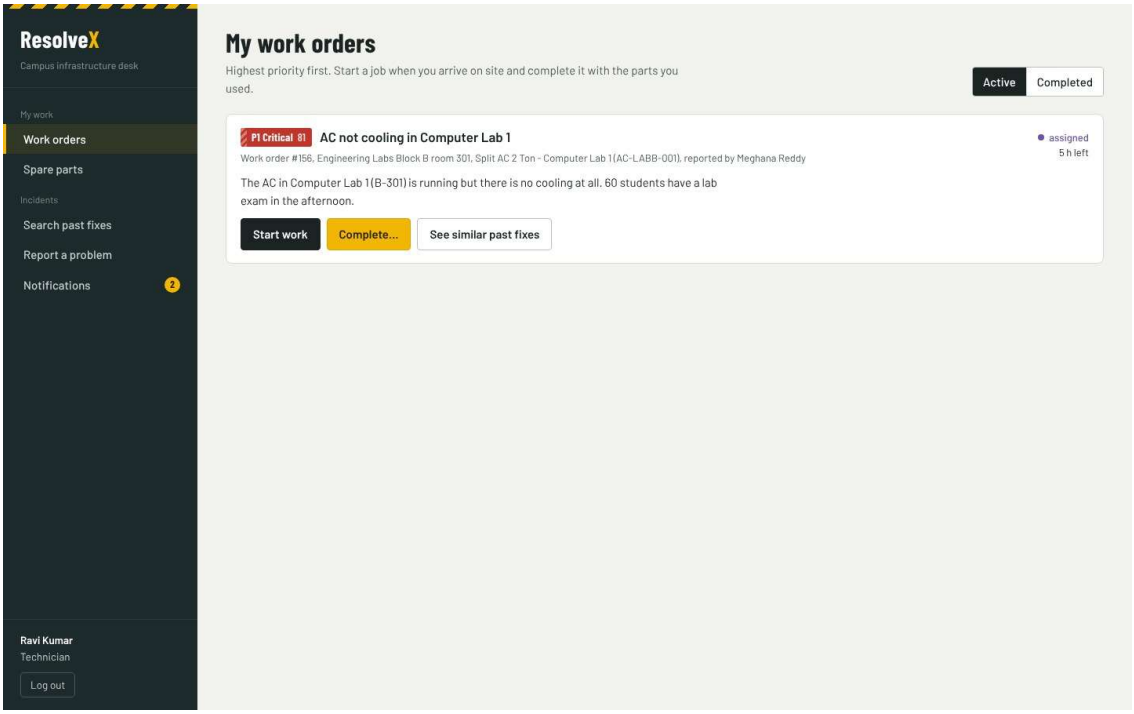


Figure 32. The technician's "My work orders" page, with the most urgent job first.

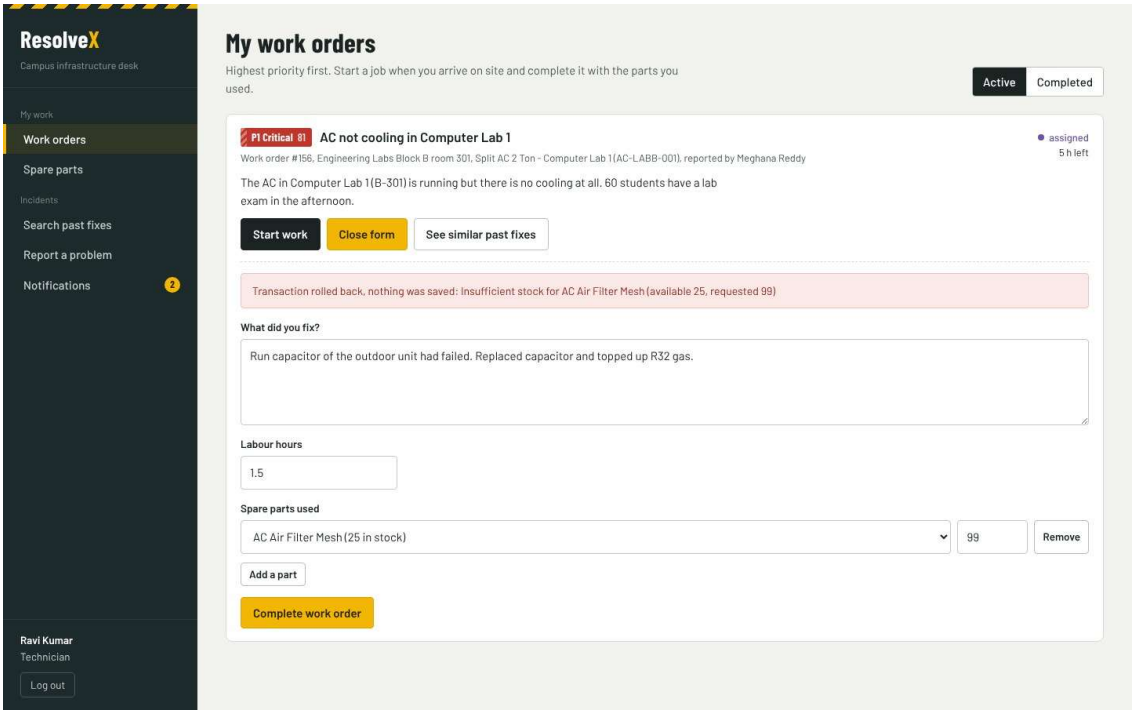


Figure 33. Trying to use 99 filters when only 25 are in stock. The transaction is rolled back and nothing is saved.

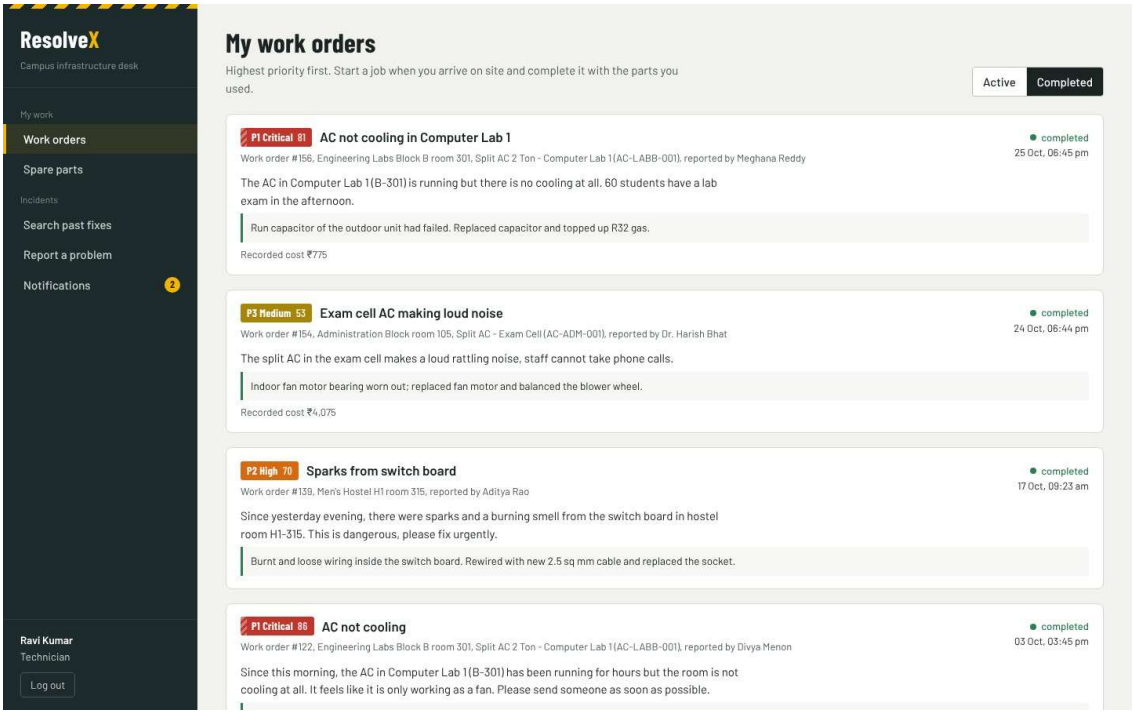


Figure 34. The same job completed with the correct quantity. Stock, cost and the incident status were all updated together.

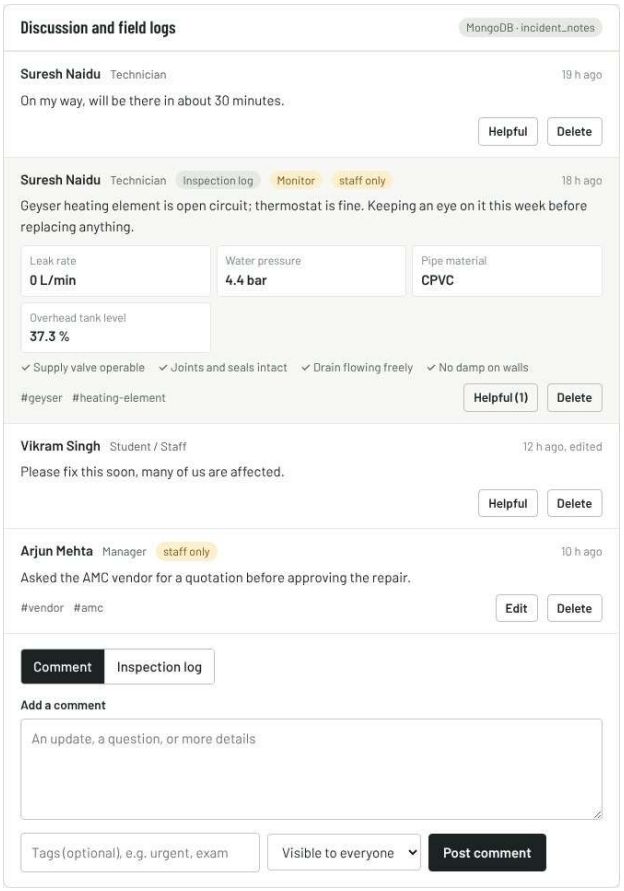


Figure 35. The discussion and inspection logs of an incident (stored in MongoDB), as seen by staff, including staff-only notes.

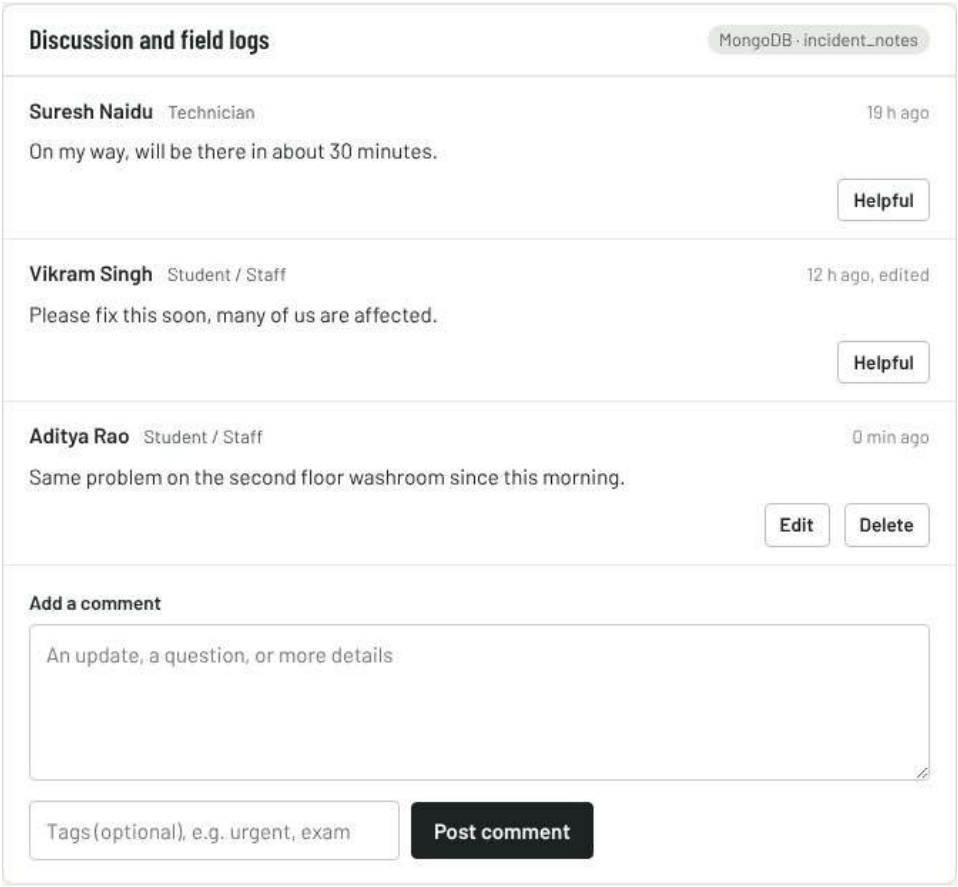


Figure 36. The same discussion as seen by a student. Staff-only notes are hidden.

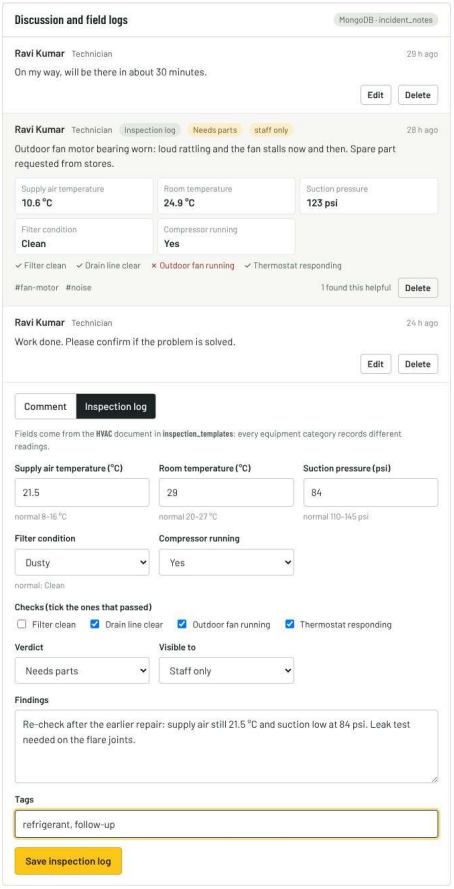


Figure 37. The inspection form. Its fields come from the HVAC template document in MongoDB.



Figure 38. A saved inspection log. Readings outside the normal range are shown in red.

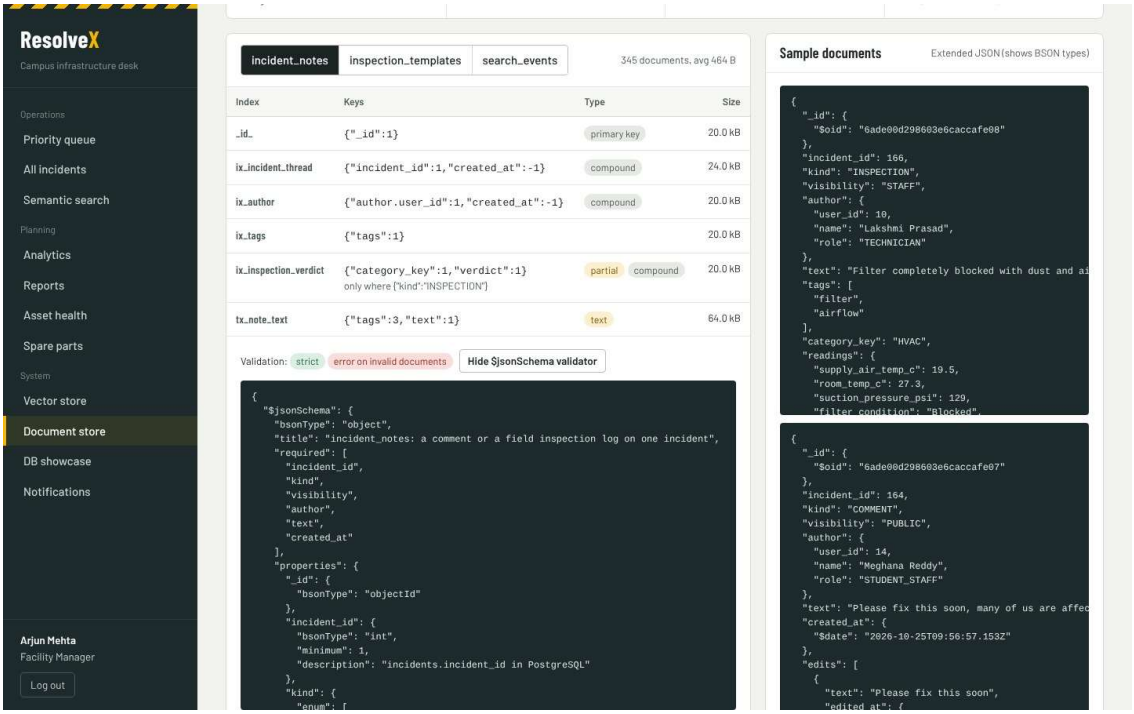


Figure 39. The `$jsonSchema` validator of the `incident_notes` collection.

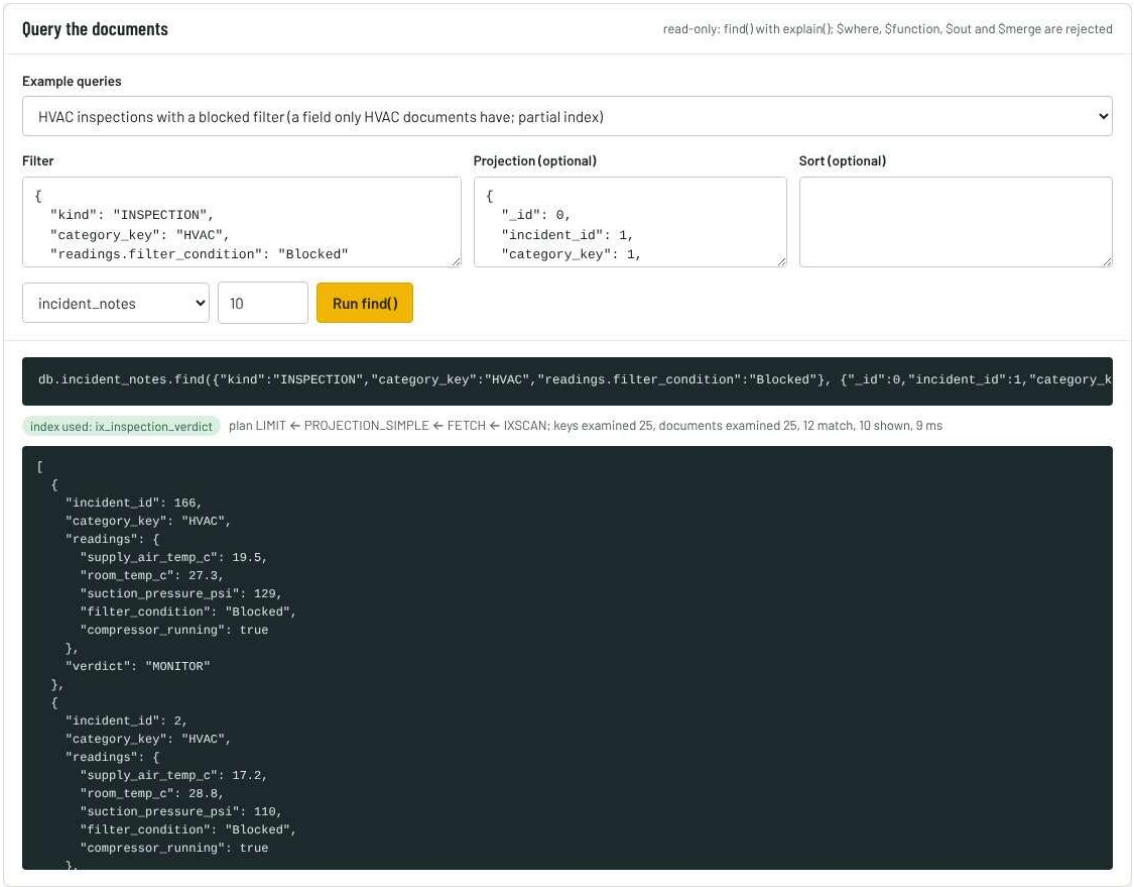


Figure 40. Running a MongoDB query with `explain()`. IXSCAN means our index was used.

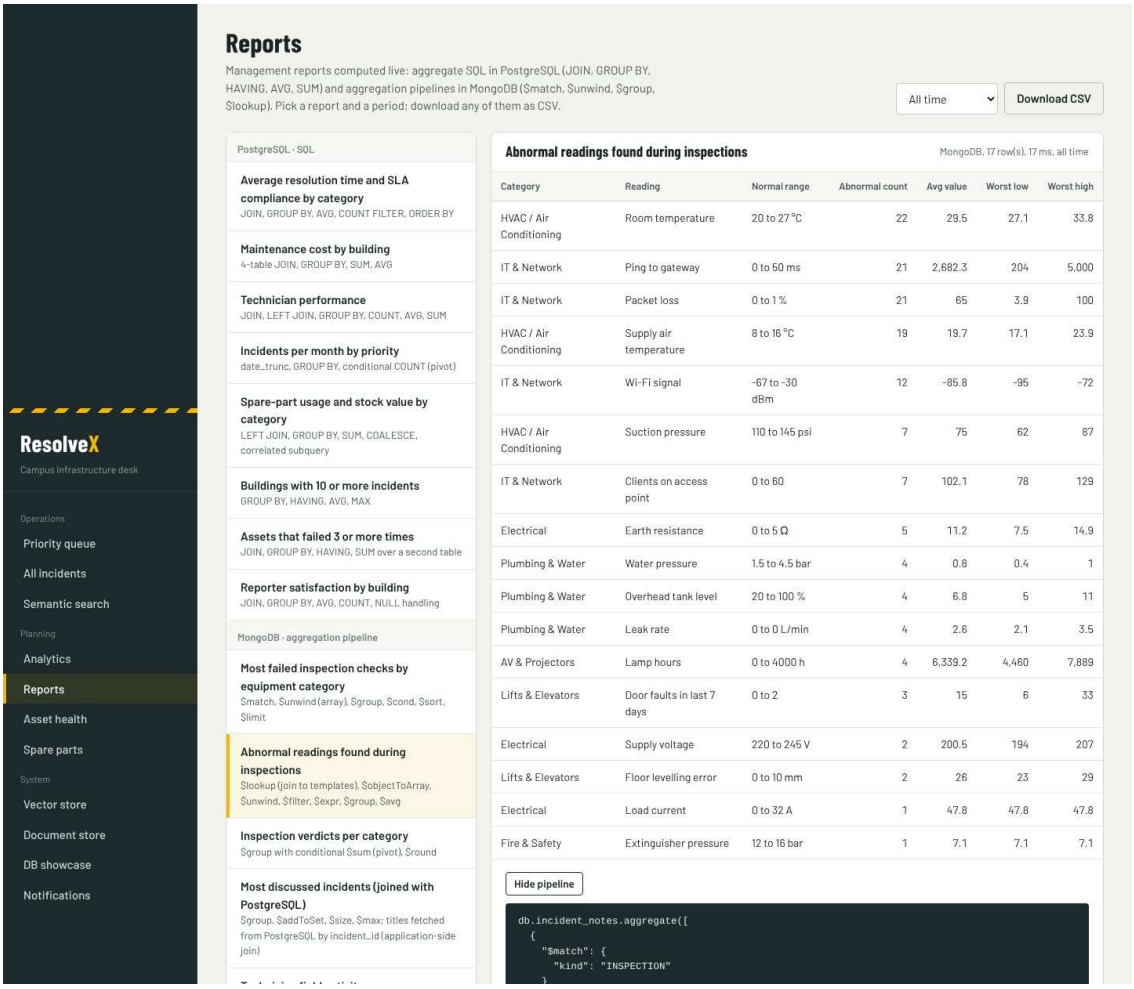


Figure 41. One of our MongoDB aggregation reports, which uses `$lookup`, `$unwind` and `$group`. (part 1 of 3)

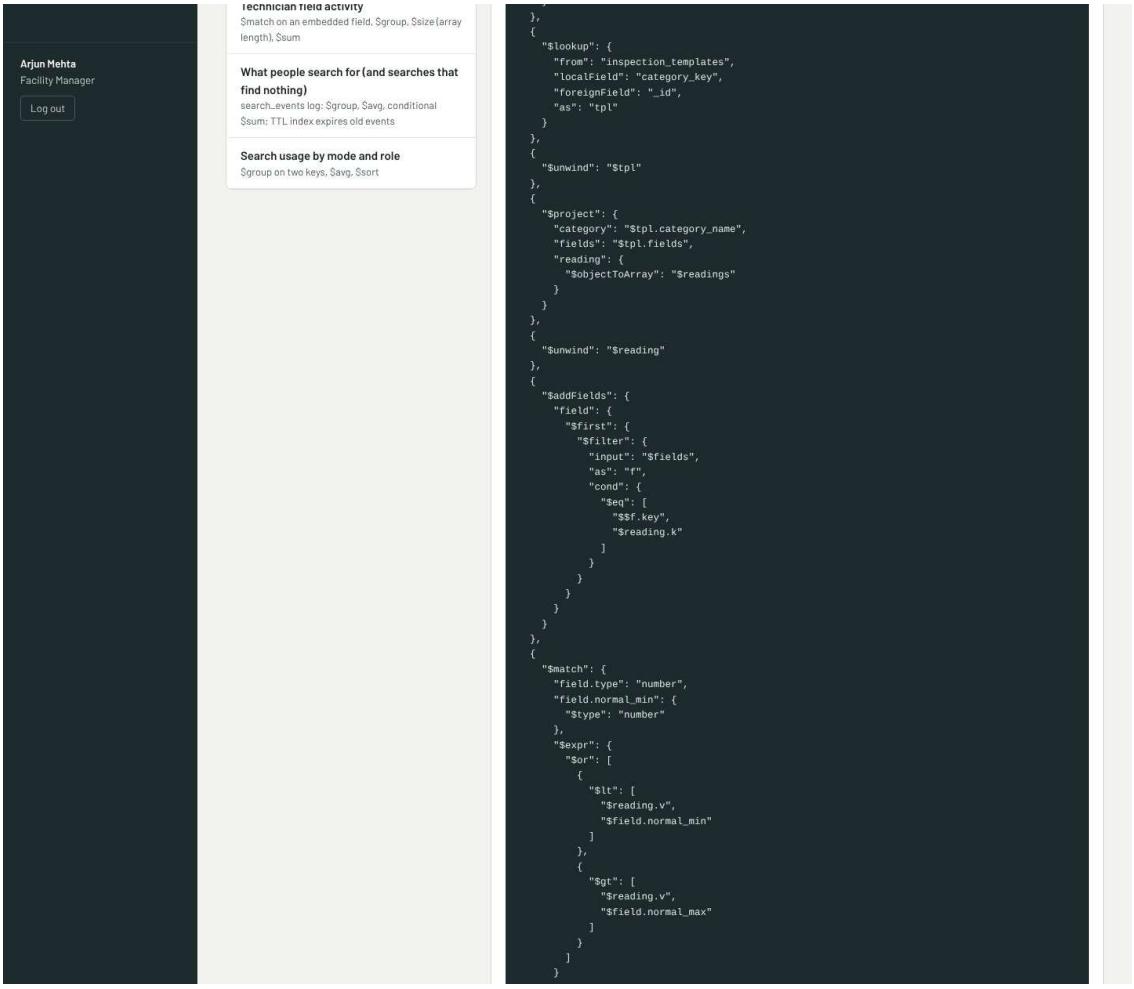
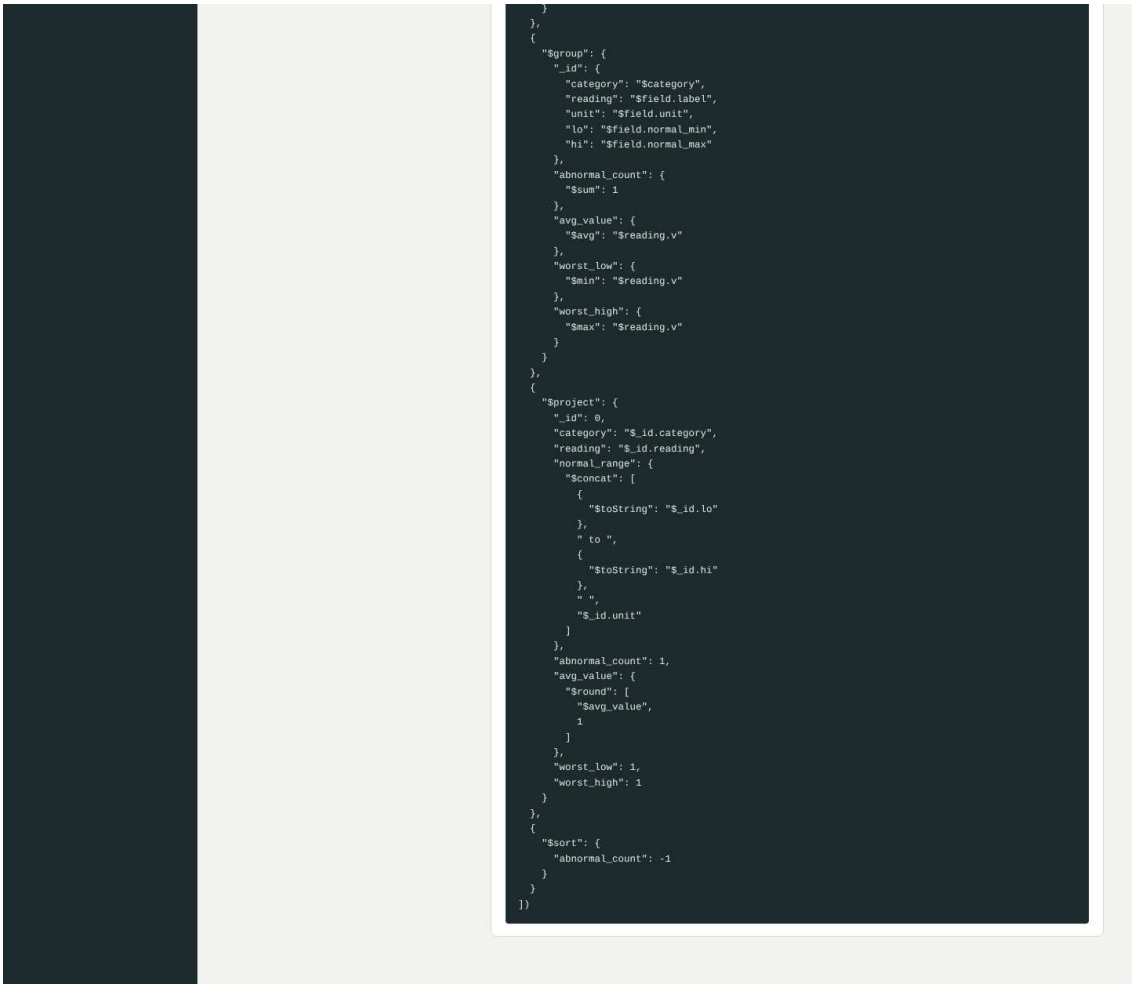


Figure 41. One of our MongoDB aggregation reports, which uses `$lookup`, `$unwind` and `$group`. (part 2 of 3)



```
    }
  },
  {
    "$group": {
      "_id": {
        "category": "$category",
        "reading": "$field.label",
        "unit": "$field.unit",
        "lo": "$field.normal_min",
        "hi": "$field.normal_max"
      },
      "abnormal_count": {
        "$sum": 1
      },
      "avg_value": {
        "$avg": "$reading.v"
      },
      "worst_low": {
        "$min": "$reading.v"
      },
      "worst_high": {
        "$max": "$reading.v"
      }
    }
  },
  {
    "$project": {
      "_id": 0,
      "category": "$_id.category",
      "reading": "$_id.reading",
      "normal_range": {
        "$concat": [
          {
            "$toString": "$_id.lo"
          },
          " to ",
          {
            "$toString": "$_id.hi"
          },
          " ",
          "$_id.unit"
        ]
      },
      "abnormal_count": 1,
      "avg_value": {
        "$round": [
          "$avg_value",
          1
        ]
      },
      "worst_low": 1,
      "worst_high": 1
    }
  },
  {
    "$sort": {
      "abnormal_count": -1
    }
  }
])
```

Figure 41. One of our MongoDB aggregation reports, which uses `$lookup`, `$unwind` and `$group`. (part 3 of 3)

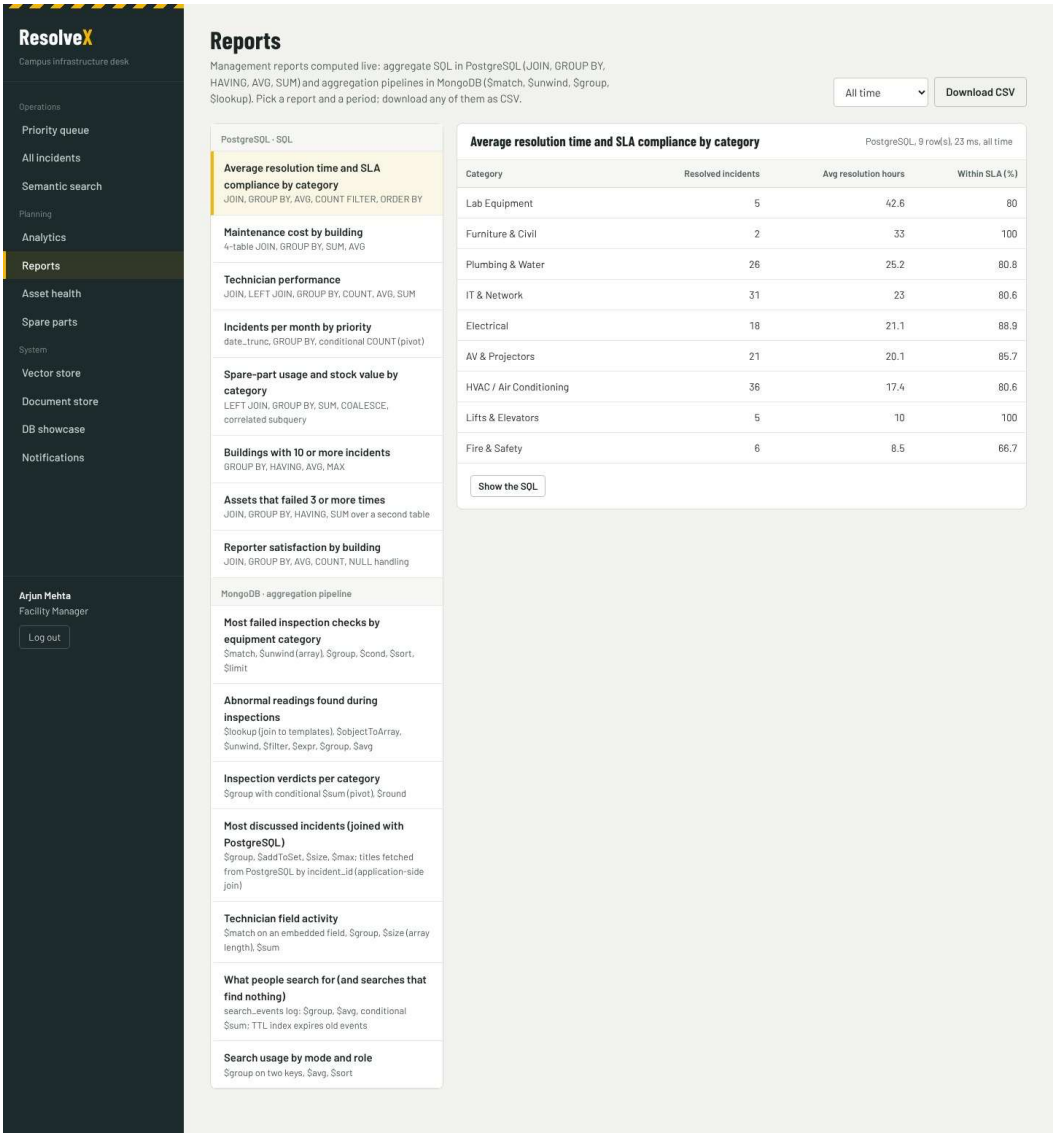


Figure 42. The Reports page, with a period filter, CSV download and a “Show the SQL” option.

11. Evaluation Rubrics – Completion Status

#	Criteria	Distribution	Status
1	Database Design	ER diagram, relational schema with keys, relationships, and constraints	Completed (Sections 3, 4)
2	Database Creation	SQL database implementation - DDL with constraints, NoSQL/Vector database creation, Technology justification	Completed (Sections 2, 5, 7)
3	SQL Operations and Queries	DML operations, Nested/correlated/Recursive SQL queries, Join SQL queries	Completed (Section 6)
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE), NoSQL/Vector feature RUD	Completed (Sections 5, 6, 7)
5	Web Application and Database Integration	Next.js interface, Backend/API and database integration	Completed (Sections 9, 10)
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	Completed (4 roles; JWT login, middleware, 65 RLS policies)
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub, Originality and individual contribution	Completed (121 automated tests)

Repository: https://github.com/SonuAkash-collab/resolvex

Our repository has the complete source code, all the SQL and MongoDB scripts, the diagrams, screenshots, tests, and the setup steps in `README.md`. We did not commit any passwords or keys. They are kept in `.env.local`, which is ignored by Git, and `.env.example` shows which settings need to be filled in.

Individual contribution

Member	Module	Main work
Sonu Akash (PES1UG24AM908)	Member 1: incident intelligence	incident reporting, priority scoring, duplicate detection, vector search, pgvector
Rishit V N S (PES1UG24AM225)	Member 2: operations	work orders, inventory, assets, analytics, authentication, security

Conclusion. Through this project we learnt that a database can do much more than just store data. In ResolveX, PostgreSQL calculates priorities and deadlines, assigns technicians safely using row locks, and completes each repair in one transaction so the data never ends up half updated. pgvector helped us catch duplicate complaints by meaning, and MongoDB let us store inspection readings that differ for every type of machine while still validating them. We also enforced security inside the database itself using 65 Row-Level Security policies. We tested

the project with 121 automated tests: 53 for SQL, 27 for MongoDB and 41 for the API and security.

Future scope. In the future we would like to let students attach photos to their reports, send email or SMS alerts when a deadline is missed, and build a mobile app for technicians to use while they are on site.